

Automated Reasoning

Artificial Intelligence

Jacopo Parretti

I Semester 2025-2026

Contents

I	6
1 Introduction	6
1.1 Why Study Automated Reasoning?	6
2 Historical Foundations and Theoretical Limits	7
2.1 The Hilbert-Turing Perspective	7
2.2 Practical Applications of Automated Reasoning	7
3 Core Problems in Automated Reasoning	7
3.1 Satisfiability Problems	8
3.2 Validity Problems	8
3.3 Logical Consequence Problems	8
4 Formal Language and Semantics	8
4.1 Logical Language: First-Order Logic	8
4.2 Signature and Universe of Discourse	8
5 Syntactic Foundations: Terms and Formulas	10
5.1 Terms: The Building Blocks	10
5.2 From Terms to Atomic Formulas	10
5.3 Literals: Positive and Negative Atomic Formulas	10
5.4 Logical Connectives	11
5.5 Formulas: The Complete Syntax	11
5.6 Quantifiers: Universality and Existence	11
5.7 Illustrative Example: Geometric Axioms	11
5.8 Closed Formulas and Variable Scope	12
II Semantic Foundations	13
6 Interpretations and Models	13
6.1 The Concept of Interpretation	13
6.2 Example: Interpreting a Formula over Natural Numbers	13
6.3 Semantic Evaluation	13
6.4 Formal Verification of Satisfaction	14
6.5 Formal Definition of Interpretation	14
7 Satisfaction Relation	15
7.1 The Satisfaction Relation	15
7.2 Interpretation of Terms	15
7.3 Satisfaction for Atomic Formulas	15
7.4 Satisfaction for Logical Connectives	15
7.5 Satisfaction for Quantified Formulas	16
7.6 Example: Quantifier Evaluation	16
8 Semantic Properties of Formulas	16
8.1 Satisfiability	17
8.2 Validity	17
8.3 Unsatisfiability	17

8.4	Invalidity	17
8.5	Relationships Between Semantic Properties	17
9	Logical Consequence	18
9.1	The Three Fundamental Query Types	18
9.2	Definition of Logical Consequence	18
9.3	The Deduction Theorem	18
9.4	Notational Ambiguity of $\models$	19
9.5	Reduction to Unsatisfiability	19
10	Refutational Reasoning	20
10.1	Proof by Contradiction	20
10.2	The Refutational Framework	20
III	Free Variables and Satisfiability	21
11	Satisfiability of Formulas with Free Variables	21
11.1	The Problem of Free Variables	21
11.2	Motivating Example	21
11.3	Semantic Evaluation with Free Variables	21
11.4	Existential Closure	22
11.5	Satisfiability for Open Formulas	22
11.6	Universal Closure and Validity	23
12	Decidability and Semidecidability in First-Order Logic	24
12.1	Computational Complexity of First-Order Logic	24
12.2	Review: Semi-Decision Procedures	24
12.3	The Need for Restricted Theories	25
13	First-Order Theories	25
13.1	Definition of a First-Order Theory	25
13.2	The Theory of Equality	25
13.3	Axioms of Equality	25
13.4	Models of a Theory	26
13.5	Satisfiability and Validity Relative to a Theory	26
13.6	Example: Satisfiability vs. $\mathcal{T}_E$ -Satisfiability	26
14	Quantifier-Free Fragments and Decidability	26
14.1	The Quantifier-Free Fragment of $\mathcal{T}_E$	26
14.2	Normal Forms	27
14.3	Conversion to DNF	28
14.4	Decision Procedure for Quantifier-Free $\mathcal{T}_E$	28
14.5	Elimination of Non-Equality Predicates	28
14.6	Treatment of Free Variables	30
15	The Congruence Closure Algorithm	30
15.1	Motivating Example	30
15.2	The Congruence Closure Decision Procedure	30
15.3	Equivalence Relations and Partitions	30
IV	Decision Procedures for Quantifier-Free Theories	32

16 The Congruence Closure Algorithm	32
16.1 Computing Congruence Closure	32
16.2 The Congruence Closure Decision Procedure	35
16.3 Examples: Manual Execution	36
17 Implementation of Congruence Closure on a DAG	39
17.1 DAG Representation	39
17.2 Node Data Structure	40
17.3 Basic Operations	40
17.4 The Complete Congruence Closure Algorithm	42
18 Worked Examples of Congruence Closure	44
18.1 Example 1: Simple Commutativity	44
18.2 Example 2: Cyclic Function Application with Disequality	45
19 Complexity Improvements and Optimizations	48
19.1 Historical Development	48
19.2 Key Optimizations	48
19.3 Implementation Note	49
19.4 Practical Impact	49
V Advanced Topics in Congruence Closure	50
20 Congruence Closure Implementation	50
20.1 DAG-Based Implementation	50
20.2 Bradley-Miller Algorithm (2007)	50
20.3 Downey-Sethi-Tarjan Improvements (1980)	50
20.4 Forbidden Set Optimization	50
20.5 Summary of Congruence Closure	51
21 The Theory of Lists	51
21.1 Signatures for List Theories	52
21.2 Axioms of the List Theory	52
21.3 Validity Examples in List Theory	53
21.4 The Theory of Possibly Empty Lists	53
21.5 Generalization to Recursive Data Structures	54
VI Theory of Lists and Recursive Data Structures	55
22 Decision Procedure for Equality and Lists	55
22.1 Problem Setting	55
22.2 Structure of the Literal Set	55
22.3 Eliminating Negative Atom Literals	55
22.4 Extending the Congruence Closure Graph	56
22.5 Post-processing for Atom Constraints	56
22.6 Example	56
VII Satisfiability Modulo Theories and Theorem Proving	57
23 Overview of SMT and ATP	57

23.1 Satisfiability Modulo Theories	57
23.2 Automated Theorem Proving	57
24 Reduction to Clausal Form	57
24.1 Basic Logical Equivalences	57
24.2 Negation Normal Form	57
24.3 Clausal Form and CNF	58
24.4 Skolemization	58
24.5 Dropping Universal Quantifiers	59
24.6 Distribution to CNF	59
24.7 Complete Example	59
VIII Theory of Arrays	60
25 The Theory of Arrays	60
25.1 Validity Example in the Theory of Arrays	62

Part I

1 Introduction

1.1 Why Study Automated Reasoning?

Automated Reasoning is a fundamental field of artificial intelligence concerned with the design and implementation of computational procedures that enable machines to solve problems formulated in logical languages. These procedures involve **logical inference and search**, allowing systems to reason in a deductive manner through mechanical processes.

Since the inception of artificial intelligence at the Dartmouth Conference in 1956, one of the core challenges has been to demonstrate machine intelligence through the automated proving of mathematical and logical theorems. This capability represents a cornerstone of symbolic AI, where knowledge is represented symbolically and stored in computer memory, and systematic procedures are applied to solve complex problems.

Theorem proving traditionally requires human-level mathematical reasoning and insight. The goal of automated reasoning is to mechanize this process, creating algorithms and systems capable of performing logical deduction without human intervention.

A logical language consists of formal symbols and syntactic rules. Within the symbolic approach to artificial intelligence, knowledge is encoded using these symbols, which are stored in computer memory. The system then applies a sequence of mechanical steps to manipulate these symbols and arrive at solutions to the given problems.

2 Historical Foundations and Theoretical Limits

2.1 The Hilbert-Turing Perspective

The foundations of automated reasoning can be traced to fundamental questions in mathematical logic and computability. Since Alan Turing’s seminal work on *Turing machines* in 1936, researchers have pursued David Hilbert’s 1900 challenge: is it possible to develop a mechanical procedure—executed deterministically by a human computer—capable of solving any mathematical problem?

The Entscheidungsproblem: A central question that emerged from this pursuit asks whether it is possible to define a mechanical method to determine the validity of formulas in first-order logic.

Turing demonstrated that the halting problem is undecidable using Turing machines. This fundamental result can be reduced to the validity problem in first-order logic, providing a negative answer to Hilbert’s challenge. This undecidability result forms one of the core theoretical foundations of automated reasoning, establishing fundamental limits on what can be mechanically computed.

2.2 Practical Applications of Automated Reasoning

Automated reasoning systems find application across multiple domains requiring formal verification and systematic problem-solving:

- **Planning:** Given an initial state, goal state, and formal model of agent actions, determine a sequence of moves that transforms the initial state into the goal state (utilizing SAT solvers)
- **Scheduling:** Generate temporal arrangements of tasks that satisfy complex temporal and resource constraints
- **Software Analysis and Verification:** Formal methods for ensuring program correctness, including:
 - Provable privacy guarantees in security protocols
 - Verification of distributed algorithms
 - Analysis of randomized algorithms and their probabilistic properties
- **Hybrid Approaches:** Integration of automated reasoning with machine learning techniques for enhanced problem-solving capabilities

These applications demonstrate the practical utility of automated reasoning in solving complex real-world problems that require rigorous logical analysis and systematic exploration of solution spaces.

3 Core Problems in Automated Reasoning

Automated reasoning fundamentally addresses three interconnected classes of decision problems in mathematical logic, each representing a distinct challenge in determining the truth properties of logical formulas.

The Three Core Problems in Automated Reasoning

1. **Satisfiability (SAT):** Does there exist a model?
2. **Validity:** Is the formula true in all models?

3. Logical Consequence: Does the conclusion follow from premises?

3.1 Satisfiability Problems

A satisfiability problem (SAT) concerns determining whether a given logical formula has a truth assignment that makes it true. Formally, for a formula ϕ with free variables, we seek to determine if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \phi$. This represents the existence problem—does there exist a model making the formula true?

3.2 Validity Problems

In contrast to satisfiability, validity problems ask whether a formula holds true under all possible interpretations. A formula ϕ is valid (denoted $\models \phi$) if every possible truth assignment to its variables results in ϕ evaluating to true. This corresponds to the universal quantification over all possible models—the formula must hold in every conceivable scenario.

3.3 Logical Consequence Problems

The logical consequence problem represents a generalization of validity, asking whether a conclusion necessarily follows from given premises. Given a set of assumptions $\Gamma = \{\phi_1, \phi_2, \dots, \phi_n\}$ and a conjecture ψ , we determine if $\Gamma \models \psi$ (i.e., if ψ is true in every model where all formulas in Γ are true). This captures the essence of logical deduction—what conclusions are forced by the given information.

These three problems form the foundation of automated reasoning systems, with satisfiability serving as the most tractable and finding widespread application in practical problem-solving contexts.

4 Formal Language and Semantics

4.1 Logical Language: First-Order Logic

The formal language employed in automated reasoning is first-order logic (FOL), a substantial extension of propositional logic that incorporates quantification over individual elements.

First-Order Logic Components

First-order logic achieves greater expressive power by introducing:

- *Individual variables* ranging over domain elements
- *Function symbols* for constructing complex terms
- *Predicate symbols* for expressing relations between elements
- *Quantifiers* ($\forall, \exists$) enabling statements about all or some elements

This enhanced expressiveness allows first-order logic to formalize mathematical structures, relationships, and properties in a manner impossible in propositional logic, which can only express truth-functional combinations of atomic propositions.

4.2 Signature and Universe of Discourse

A signature Σ provides the vocabulary for interpreting formulas within a specific domain. Formally, a signature is a triple $\Sigma = \langle S, \mathcal{F}, \mathcal{P} \rangle$ where:

- S is a set of sorts (disjoint domains of interpretation)

- $\mathcal{F}$ is a set of function symbols, each with an arity specifying the number of arguments
- $\mathcal{P}$ is a set of predicate symbols, which we use to write formulas, each with an associated arity

The signature defines the syntactic elements available for constructing formulas, while the universe of discourse (or domain) provides the semantic interpretation. Sorts correspond to types in programming languages, enabling the formal specification of heterogeneous domains with different categories of objects.

The signature $\Sigma = \langle S, \mathcal{F}, \mathcal{P} \rangle$ can be further decomposed to provide a complete syntactic framework. The set of function symbols $\mathcal{F}$ is typically partitioned into two distinct subsets:

- **Constant symbols** $C \subseteq \mathcal{F}$: These are function symbols of arity 0, representing named individuals in the domain. Constants have fixed interpretations and serve as proper names for specific domain elements.
- **Proper function symbols** $\mathcal{F} \setminus C$: These represent actual functions mapping tuples of domain elements to domain elements, each with arity $n \geq 1$.

Formally, a function symbol $f \in \mathcal{F}$ has type $s_1 \times s_2 \times \cdots \times s_n \rightarrow s$, where $s_1, \dots, s_n, s \in S$ are sorts indicating the domain and codomain of the function. For example:

- $c : \rightarrow \text{int}$ (a constant symbol denoting a specific integer)
- $p : \rightarrow \text{color}$ (a constant symbol denoting a specific color)
- $f : \text{int} \times \text{int} \rightarrow \text{int}$ (a binary function like addition)

This decomposition enables precise specification of both individual elements (via constants) and functional relationships (via proper function symbols) within the formal language.

The complete signature structure $\langle S, C \cup (\mathcal{F} \setminus C) \cup \mathcal{P} \rangle$ provides the full syntactic foundation for expressing complex logical statements about structured domains.

Constants can be viewed as function symbols of arity 0—functions that require no arguments and directly denote specific domain elements. Additionally, to construct quantified formulas and express generality, we require a countably infinite set of variables $X = \{x, y, z, \dots\}$ disjoint from the signature symbols.

Thus, the function symbols may be comprehensively categorized as $\mathcal{F} = C \cup (\mathcal{F} \setminus C) \cup \mathcal{P}$, where $\mathcal{P}$ represents the set of predicate symbols. Each predicate symbol $p \in \mathcal{P}$ has arity $n \geq 0$ and type $s_1 \times s_2 \times \cdots \times s_n$, where the final sort represents the truth value (typically a boolean sort).

Predicate symbols differ fundamentally from function symbols in that they represent relations rather than functions—they evaluate to truth values rather than domain elements. For example:

- $\text{even} : \text{int} \rightarrow \text{bool}$ (unary predicate testing parity)
- $\text{less} : \text{int} \times \text{int} \rightarrow \text{bool}$ (binary relation for ordering)

The complete signature structure $\langle S, C \cup (\mathcal{F} \setminus C) \cup \mathcal{P} \rangle$ provides the full syntactic foundation for expressing complex logical statements about structured domains.

Constants can be viewed as function symbols of arity 0—functions that require no arguments and directly denote specific domain elements. Additionally, to construct quantified formulas and express generality, we require a countably infinite set of variables $X = \{x, y, z, \dots\}$ disjoint from the signature symbols.

5 Syntactic Foundations: Terms and Formulas

5.1 Terms: The Building Blocks

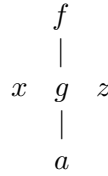
The most fundamental syntactic objects in first-order logic are **terms**, which represent individuals in the domain of discourse. The set of terms $\mathcal{T}_\Sigma(X)$ over signature Σ and variables X is inductively defined as the smallest set satisfying:

- **Constants:** If $c \in C$ is a constant symbol, then $c \in \mathcal{T}_\Sigma(X)$
- **Variables:** If $x \in X$ is a variable, then $x \in \mathcal{T}_\Sigma(X)$
- **Function application:** If $f \in \mathcal{F}$ is a function symbol of arity $n \geq 1$ and $t_1, \dots, t_n \in \mathcal{T}_\Sigma(X)$ are terms, then $f(t_1, \dots, t_n) \in \mathcal{T}_\Sigma(X)$

Terms can be conceptualized as finite trees where:

- Leaf nodes are constants or variables
- Internal nodes are function symbols applied to subterms

For example, the term $f(x, g(a), z)$ has the tree representation:



Terms may contain subterms, which correspond to subtrees in this representation. Complex terms are constructed recursively from simpler subterms, reflecting the hierarchical nature of the syntactic structure.

5.2 From Terms to Atomic Formulas

Having established the notion of terms as representations of individuals, we now introduce atomic formulas to express relationships between these individuals. An **atomic formula** (or **atom**) is constructed by applying a predicate symbol to an appropriate number of terms.

Formally, if $P \in \mathcal{P}$ is a predicate symbol of arity $n \geq 0$ and $t_1, \dots, t_n \in \mathcal{T}_\Sigma(X)$ are terms, then $P(t_1, \dots, t_n)$ is an atomic formula. When $n = 0$, the atomic formula reduces to the predicate symbol itself P , which can be interpreted as a propositional constant.

The truth value of an atomic formula $P(t_1, \dots, t_n)$ depends on the interpretation of the predicate P and the denotations of the terms $t_1, \dots, t_n$ in a given model.

5.3 Literals: Positive and Negative Atomic Formulas

A **literal** is either an atomic formula (positive literal) or its negation (negative literal). Formally, if ϕ is an atomic formula, then:

- ϕ is a positive literal
- $\neg\phi$ is a negative literal

Literals represent the basic building blocks for constructing more complex logical formulas and play a crucial role in automated reasoning systems, particularly in resolution-based theorem proving.

5.4 Logical Connectives

Logical connectives provide the means to construct complex formulas from simpler ones. These operators enable the expression of compound logical statements through systematic combination of atomic formulas and literals.

The primary logical connectives include:

- **Negation** ($\neg$): Expresses logical denial or contradiction
- **Conjunction** ($\wedge$): Represents logical conjunction (AND)
- **Disjunction** ($\vee$): Represents logical disjunction (OR)
- **Implication** ($\rightarrow$): Expresses logical implication (IF...THEN)
- **Biconditional** ($\leftrightarrow$): Represents logical equivalence (IF AND ONLY IF)

5.5 Formulas: The Complete Syntax

The set of **formulas** (or **well-formed formulas**) $\mathcal{F}_\Sigma(X)$ over signature Σ and variables X is inductively defined as the smallest set satisfying:

- **Atomic formulas**: If $P \in \mathcal{P}$ is a predicate symbol of arity $n \geq 0$ and $t_1, \dots, t_n \in \mathcal{T}_\Sigma(X)$, then $P(t_1, \dots, t_n) \in \mathcal{F}_\Sigma(X)$
- **Negation**: If $\phi \in \mathcal{F}_\Sigma(X)$, then $(\neg\phi) \in \mathcal{F}_\Sigma(X)$
- **Binary connectives**: If $\phi, \psi \in \mathcal{F}_\Sigma(X)$, then:
 - $(\phi \wedge \psi) \in \mathcal{F}_\Sigma(X)$ (conjunction)
 - $(\phi \vee \psi) \in \mathcal{F}_\Sigma(X)$ (disjunction)
 - $(\phi \rightarrow \psi) \in \mathcal{F}_\Sigma(X)$ (implication)
 - $(\phi \leftrightarrow \psi) \in \mathcal{F}_\Sigma(X)$ (biconditional)

This inductive definition ensures that all syntactically valid logical expressions can be systematically constructed from the basic building blocks of atomic formulas through the application of logical connectives.

5.6 Quantifiers: Universality and Existence

To complete the syntax of first-order logic, we introduce quantifiers that express generality and existence over the domain of discourse:

- **Universal quantifier** ($\forall$): If $\phi \in \mathcal{F}_\Sigma(X)$ is a formula and $x \in X$ is a variable, then $(\forall x \phi) \in \mathcal{F}_\Sigma(X)$
- **Existential quantifier** ($\exists$): If $\phi \in \mathcal{F}_\Sigma(X)$ is a formula and $x \in X$ is a variable, then $(\exists x \phi) \in \mathcal{F}_\Sigma(X)$

Quantifiers bind variables within their scope, enabling the expression of properties that hold for all elements of the domain (universal quantification) or for at least one element (existential quantification).

5.7 Illustrative Example: Geometric Axioms

Consider the following first-order formula expressing that any two distinct points determine a unique line:

$$\forall x \forall y ((P(x) \wedge P(y) \wedge x \neq y) \rightarrow \exists z (L(z) \wedge Q(z, x, y) \wedge \forall w (L(w) \wedge Q(w, x, y) \rightarrow w = z)))$$

Where:

- $P(x)$ denotes that x is a point
- $L(z)$ denotes that z is a line
- $Q(z, x, y)$ denotes that line z passes through points x and y
- $x \neq y$ expresses that points x and y are distinct

This axiom formalizes the geometric principle that given any two distinct points in space, there exists exactly one line passing through both points, capturing a fundamental property of Euclidean geometry within the framework of first-order logic.

5.8 Closed Formulas and Variable Scope

A **closed formula** (or **sentence**) is a formula containing no free variables—all variables are bound by quantifiers. A variable is **free** in a formula if it appears outside the scope of any quantifier that binds it; conversely, a variable is **bound** if it appears within the scope of a quantifier.

Formulas may contain both free and bound occurrences of variables. For proper semantic interpretation, variables must be **standardized apart**, meaning that no variable appears both free and bound in the same formula, and distinct quantifiers use distinct bound variables.

Part II

Semantic Foundations

6 Interpretations and Models

6.1 The Concept of Interpretation

To assign meaning to syntactic formulas in first-order logic, we require a formal notion of **interpretation**. An interpretation $\mathcal{I}$ provides a semantic mapping from the syntactic elements of a signature to concrete mathematical objects.

Interpretation Components

In the unsorted case, an interpretation consists of:

- A non-empty set D called the **domain** (or universe of discourse)
- An interpretation function Φ that maps:
 - Each predicate symbol $P \in \mathcal{P}$ of arity n to a subset $\Phi(P) \subseteq D^n$ (representing an n -ary relation over the domain)
 - Each function symbol $f \in \mathcal{F}$ of arity n to a function $\Phi(f) : D^n \rightarrow D$
 - Each constant symbol $c \in C$ to a specific element $\Phi(c) \in D$

The interpretation function Φ bridges the gap between syntax and semantics, transforming abstract symbols into concrete mathematical entities.

6.2 Example: Interpreting a Formula over Natural Numbers

Consider the first-order formula:

$$\forall x (\neg P(x) \rightarrow P(f(x)))$$

This formula contains:

- A unary predicate symbol P
- A unary function symbol f
- A universally quantified variable x

Since the formula is closed (contains no free variables), its truth value depends entirely on the chosen interpretation.

Constructing an interpretation: Let us define an interpretation $\mathcal{I} = \langle D, \Phi \rangle$ where:

- **Domain:** $D = \mathbb{N}$ (the set of natural numbers)
- **Function interpretation:** $\Phi(f) : \mathbb{N} \rightarrow \mathbb{N}$ is the successor function, defined by $\Phi(f)(n) = n + 1$ for all $n \in \mathbb{N}$
- **Predicate interpretation:** $\Phi(P) = \{0, 2, 4, 6, \dots\} = \{2k \mid k \in \mathbb{N}\}$ (the set of even natural numbers)

6.3 Semantic Evaluation

Under this interpretation $\mathcal{I}$, the formula $\forall x (\neg P(x) \rightarrow P(f(x)))$ receives the following semantic reading:

For all natural numbers x , if x is not even, then the successor of x is even.

To determine whether this formula is true in interpretation $\mathcal{I}$, we must verify whether the implication holds for every element of the domain:

- If x is odd (i.e., $x \notin \Phi(P)$), then $x + 1$ must be even (i.e., $x + 1 \in \Phi(P)$)

This statement is indeed true in the natural numbers: the successor of any odd number is even. Therefore, the formula is **satisfied** by interpretation $\mathcal{I}$, and we write $\mathcal{I} \models \forall x (\neg P(x) \rightarrow P(f(x)))$.

6.4 Formal Verification of Satisfaction

To formally verify that $\mathcal{I} \models \forall x (\neg P(x) \rightarrow P(f(x)))$, we must check that for all $n \in \mathbb{N}$, the interpretation satisfies the implication with the substitution $\{x \leftarrow n\}$.

By the semantics of implication, $\mathcal{I}$ satisfies $(\neg P(x) \rightarrow P(f(x)))$ with assignment $\{x \leftarrow n\}$ if and only if:

- Either $\mathcal{I}$ does not satisfy $\neg P(x)$ with $\{x \leftarrow n\}$ (i.e., $\mathcal{I}$ satisfies $P(x)$ with $\{x \leftarrow n\}$), or
- $\mathcal{I}$ satisfies $P(f(x))$ with $\{x \leftarrow n\}$

Equivalently, for all $n \in \mathbb{N}$, either $n \in \Phi(P)$ or $\Phi(f)(n) \in \Phi(P)$.

Truth condition for atomic formulas: An atomic formula $P(t)$ is true under interpretation $\mathcal{I}$ if and only if the interpretation of the term t belongs to the interpretation of the predicate $\Phi(P)$.

In our example, for all $n \in \mathbb{N}$:

- Either $n \in \Phi(P)$ (meaning n is even), or
- $\Phi(f)(n) = n + 1 \in \Phi(P)$ (meaning the successor of n is even)

This condition holds for all natural numbers, confirming that the formula is satisfied by the interpretation.

6.5 Formal Definition of Interpretation

Having illustrated the concept through examples, we now provide the complete formal definition of an interpretation in first-order logic.

An **interpretation** $\mathcal{I}$ consists of a triple $\mathcal{I} = \langle D, \Phi, \beta \rangle$ where:

- **Domain:** D is a non-empty set representing the universe of discourse
- **Interpretation function** Φ maps signature symbols to semantic objects:
 - For each constant symbol $c \in C$: $\Phi(c) = d \in D$
 - For each function symbol $f \in \mathcal{F}$ of arity $n \geq 1$: $\Phi(f) : D^n \rightarrow D$, where D^n denotes the n -fold Cartesian product $D \times D \times \dots \times D$
 - For each predicate symbol $P \in \mathcal{P}$ of arity $n \geq 0$: $\Phi(P) \subseteq D^n$ (an n -ary relation over D)
- **Variable assignment** β maps each variable to a domain element:
 - $\beta : X \rightarrow D$ such that $\beta(x) = d \in D$ for all variables $x \in X$

Assignment update notation: Given an assignment β and a substitution $x \leftarrow d$, we define the updated assignment $\beta[x \leftarrow d]$ as:

$$\beta[x \leftarrow d](y) = \begin{cases} d & \text{if } y = x \\ \beta(y) & \text{otherwise} \end{cases}$$

This notation allows us to formally express the evaluation of formulas with different variable bindings, which is essential for defining the semantics of quantified formulas.

7 Satisfaction Relation

7.1 The Satisfaction Relation

Having defined interpretations and variable assignments, we now formalize the central semantic concept: when does an interpretation **satisfy** a formula? The satisfaction relation, denoted $\mathcal{I} \models_{\beta} \phi$, specifies the conditions under which a formula ϕ is true in interpretation $\mathcal{I}$ with variable assignment β .

Let F and G denote arbitrary formulas. The satisfaction relation is defined inductively on the structure of formulas.

7.2 Interpretation of Terms

Before defining satisfaction for formulas, we must specify how terms are interpreted. Given an interpretation $\mathcal{I} = \langle D, \Phi, \beta \rangle$, the interpretation of a term t , denoted $\Phi_{\beta}(t)$, is defined recursively:

- **Constants:** $\Phi_{\beta}(c) = \Phi(c) \in D$
- **Variables:** $\Phi_{\beta}(x) = \beta(x) \in D$
- **Function application:** $\Phi_{\beta}(f(t_1, \dots, t_n)) = \Phi(f)(\Phi_{\beta}(t_1), \dots, \Phi_{\beta}(t_n)) \in D$

Each term evaluates to a specific element of the domain under a given interpretation.

7.3 Satisfaction for Atomic Formulas

Base case (atomic formulas): An interpretation $\mathcal{I}$ satisfies an atomic formula $P(t_1, \dots, t_n)$ under assignment β if and only if the tuple of interpreted terms belongs to the interpretation of the predicate:

$$\mathcal{I} \models_{\beta} P(t_1, \dots, t_n) \quad \text{iff} \quad (\Phi_{\beta}(t_1), \dots, \Phi_{\beta}(t_n)) \in \Phi(P)$$

7.4 Satisfaction for Logical Connectives

The satisfaction relation extends to compound formulas through the following inductive clauses. Note that all satisfaction relations are parameterized by the variable assignment β , written as $\models_{\beta}$.

- **Negation:**

$$\mathcal{I} \models_{\beta} \neg F \quad \text{iff} \quad \mathcal{I} \not\models_{\beta} F$$

- **Conjunction:**

$$\mathcal{I} \models_{\beta} (F \wedge G) \quad \text{iff} \quad \mathcal{I} \models_{\beta} F \text{ and } \mathcal{I} \models_{\beta} G$$

- **Disjunction:**

$$\mathcal{I} \models_{\beta} (F \vee G) \quad \text{iff} \quad \mathcal{I} \models_{\beta} F \text{ or } \mathcal{I} \models_{\beta} G$$

- **Implication:**

$$\mathcal{I} \models_{\beta} (F \rightarrow G) \quad \text{iff} \quad \mathcal{I} \not\models_{\beta} F \text{ or } \mathcal{I} \models_{\beta} G$$

Equivalently: if $\mathcal{I} \models_{\beta} F$ then $\mathcal{I} \models_{\beta} G$

- **Biconditional:**

$$\mathcal{I} \models_{\beta} (F \leftrightarrow G) \quad \text{iff} \quad \mathcal{I} \models_{\beta} F \text{ if and only if } \mathcal{I} \models_{\beta} G$$

7.5 Satisfaction for Quantified Formulas

Quantifiers introduce variable bindings that range over the entire domain:

- **Universal quantification:**

$$\mathcal{I} \models_{\beta} \forall x G \quad \text{iff} \quad \text{for all } d \in D, \mathcal{I} \models_{\beta[x \leftarrow d]} G$$

The formula $\forall x G$ is satisfied if G is satisfied under every possible assignment to x .

- **Existential quantification:**

$$\mathcal{I} \models_{\beta} \exists x G \quad \text{iff} \quad \text{there exists } d \in D \text{ such that } \mathcal{I} \models_{\beta[x \leftarrow d]} G$$

The formula $\exists x G$ is satisfied if G is satisfied under at least one assignment to x .

This completes the inductive definition of the satisfaction relation, providing a complete semantics for first-order logic.

7.6 Example: Quantifier Evaluation

Consider the formula $\forall x \exists y R(x, y)$, which can be read as "for all x , there exists y such that $x < y$ ".

Let $\mathcal{I} = \langle \mathbb{N}, \Phi \rangle$ where $\Phi(R) = \{(n, m) \in \mathbb{N} \times \mathbb{N} \mid n < m\}$ (the standard ordering on natural numbers).

To verify $\mathcal{I} \models \forall x \exists y R(x, y)$, we apply the semantic definitions:

$$\begin{aligned} \mathcal{I} \models \forall x \exists y R(x, y) & \quad \text{iff} \quad \text{for all } n \in \mathbb{N}, \mathcal{I} \models_{\beta[x \leftarrow n]} \exists y R(x, y) \\ & \quad \text{iff} \quad \text{for all } n \in \mathbb{N}, \text{ there exists } m \in \mathbb{N} \text{ such that } \mathcal{I} \models_{\beta[x \leftarrow n, y \leftarrow m]} R(x, y) \\ & \quad \text{iff} \quad \text{for all } n \in \mathbb{N}, \text{ there exists } m \in \mathbb{N} \text{ such that } (n, m) \in \Phi(R) \\ & \quad \text{iff} \quad \text{for all } n \in \mathbb{N}, \text{ there exists } m \in \mathbb{N} \text{ such that } n < m \end{aligned}$$

This statement is **false** in $\mathbb{N}$ because there is no natural number greater than all natural numbers. However, it would be true in $\mathbb{Z}$ or $\mathbb{Q}$, demonstrating how truth depends on the chosen interpretation.

8 Semantic Properties of Formulas

Having established the satisfaction relation, we now define fundamental semantic properties that classify formulas according to their truth behavior across different interpretations.

Semantic Properties of Formulas

Satisfiability

A formula F is **satisfiable** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.

$$F \text{ is satisfiable} \Leftrightarrow \exists \mathcal{I} : \mathcal{I} \models F$$

Validity

A formula F is **valid** (or a **tautology**) if for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models F$.

$$F \text{ is valid} \Leftrightarrow \forall \mathcal{I} : \mathcal{I} \models F$$

Unsatisfiability

A formula F is **unsatisfiable** if there exists no interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.

$$F \text{ is unsatisfiable} \Leftrightarrow \nexists \mathcal{I} : \mathcal{I} \models F$$

Invalidity

A formula F is **invalid** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \not\models F$.

$$F \text{ is invalid} \Leftrightarrow \exists \mathcal{I} : \mathcal{I} \not\models F$$

8.1 Satisfiability

A formula F is **satisfiable** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$. In this case, we say that $\mathcal{I}$ is a **model** of F .

8.2 Validity

We denote validity by $\models F$.

8.3 Unsatisfiability

Unsatisfiable formulas are false in every interpretation.

8.4 Invalidity

In this case, we say that $\mathcal{I}$ is a **countermodel** (or **counterexample**) for F .

8.5 Relationships Between Semantic Properties

These semantic properties are intimately related through negation:

Theorem 8.1 (Validity and Unsatisfiability). *A formula F is valid if and only if $\neg F$ is unsatisfiable.*

Proof. Suppose F is valid. Then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models F$, which implies $\mathcal{I} \not\models \neg F$. Therefore, $\neg F$ is unsatisfiable.

Conversely, if $\neg F$ is unsatisfiable, then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \not\models \neg F$, which implies $\mathcal{I} \models F$. Therefore, F is valid. $\square$

Theorem 8.2 (Satisfiability and Invalidity). *A formula F is satisfiable if and only if $\neg F$ is invalid.*

Proof. F is satisfiable if and only if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$, which is equivalent to $\mathcal{I} \not\models \neg F$. This is precisely the condition for $\neg F$ to be invalid. $\square$

These relationships demonstrate the duality between validity and unsatisfiability, and between satisfiability and invalidity, providing a complete characterization of the semantic landscape of first-order logic.

9 Logical Consequence

9.1 The Three Fundamental Query Types

Automated reasoning systems must be capable of resolving three fundamental types of semantic queries:

1. **Satisfiability queries:** Is formula F satisfiable?
2. **Validity queries:** Is formula F valid?
3. **Logical consequence queries:** Does F follow logically from a set of hypotheses H ?

9.2 Definition of Logical Consequence

Logical Consequence

Let $H = \{\phi_1, \phi_2, \dots, \phi_n\}$ be a set of formulas (the **hypotheses** or **premises**) and let F be a formula (the **conclusion**). We say that F is a **logical consequence** of H , written $H \models F$, if:

$$H \models F \quad \Leftrightarrow \quad \text{for all interpretations } \mathcal{I}, \text{ if } \mathcal{I} \models H \text{ then } \mathcal{I} \models F$$

In other words, F is a logical consequence of H if every model of H is also a model of F . Here, $\mathcal{I} \models H$ means that $\mathcal{I}$ satisfies all formulas in H .

9.3 The Deduction Theorem

The deduction theorem establishes a fundamental connection between logical consequence and validity, reducing the problem of checking entailment to the problem of checking validity.

Deduction Theorem

Theorem 9.1 (Deduction Theorem). *Let H be a set of formulas and F be a formula. Then:*

$$H \models F \quad \text{iff} \quad \models (H \rightarrow F)$$

where $H \rightarrow F$ denotes the implication from the conjunction of all formulas in H to F .

Proof. We prove both directions:

($\Rightarrow$) Assume $H \models F$. We must show that $H \rightarrow F$ is valid, i.e., for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models H \rightarrow F$.

Let $\mathcal{I}$ be an arbitrary interpretation. By the semantics of implication, $\mathcal{I} \models H \rightarrow F$ holds if either:

- $\mathcal{I} \not\models H$ (the antecedent is false), or
- $\mathcal{I} \models F$ (the consequent is true)

If $\mathcal{I} \not\models H$, then $\mathcal{I} \models H \rightarrow F$ trivially. If $\mathcal{I} \models H$, then by the hypothesis $H \models F$, we have $\mathcal{I} \models F$, so $\mathcal{I} \models H \rightarrow F$. In both cases, $\mathcal{I} \models H \rightarrow F$.

($\Leftarrow$) Conversely, suppose $H \rightarrow F$ is valid. Then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models H \rightarrow F$. By the semantics of implication, for all $\mathcal{I}$, either $\mathcal{I} \not\models H$ or $\mathcal{I} \models F$. Therefore, for all $\mathcal{I}$ such that $\mathcal{I} \models H$, we must have $\mathcal{I} \models F$, which is precisely the definition of $H \models F$. $\square$

9.4 Notational Ambiguity of $\models$

The symbol $\models$ is used in three distinct but related contexts:

- **Satisfaction:** $\mathcal{I} \models F$ means interpretation $\mathcal{I}$ satisfies formula F
- **Validity:** $\models F$ means formula F is valid (satisfied by all interpretations)
- **Entailment:** $H \models F$ means formula F is a logical consequence of the set of formulas H

While this overloading may initially seem confusing, these three uses are semantically coherent and represent different aspects of the same fundamental semantic relation.

9.5 Reduction to Unsatisfiability

A fundamental result connects logical consequence to unsatisfiability, providing another characterization of entailment.

Key Theorem: Reduction to Unsatisfiability

Theorem 9.2 (Logical Consequence via Unsatisfiability). *Let H be a set of formulas and F be a formula. Then:*

$$H \models F \quad \text{iff} \quad H \cup \{\neg F\} \text{ is unsatisfiable}$$

Proof. We prove both directions:

($\Rightarrow$) Assume $H \models F$. We must show that $H \cup \{\neg F\}$ is unsatisfiable.

Suppose for contradiction that there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models H \cup \{\neg F\}$. Then $\mathcal{I} \models H$ and $\mathcal{I} \models \neg F$, which implies $\mathcal{I} \not\models F$. However, since $H \models F$ and $\mathcal{I} \models H$, we must have $\mathcal{I} \models F$, which is a contradiction. Therefore, $H \cup \{\neg F\}$ is unsatisfiable.

($\Leftarrow$) Assume $H \cup \{\neg F\}$ is unsatisfiable. We must show that $H \models F$.

Let $\mathcal{I}$ be any interpretation such that $\mathcal{I} \models H$. Since $H \cup \{\neg F\}$ is unsatisfiable, there is no interpretation that satisfies both H and $\neg F$. Therefore, $\mathcal{I} \not\models \neg F$, which implies $\mathcal{I} \models F$. Since this holds for all interpretations satisfying H , we conclude that $H \models F$. $\square$

10 Refutational Reasoning

10.1 Proof by Contradiction

Refutational Reasoning (Proof by Contradiction)

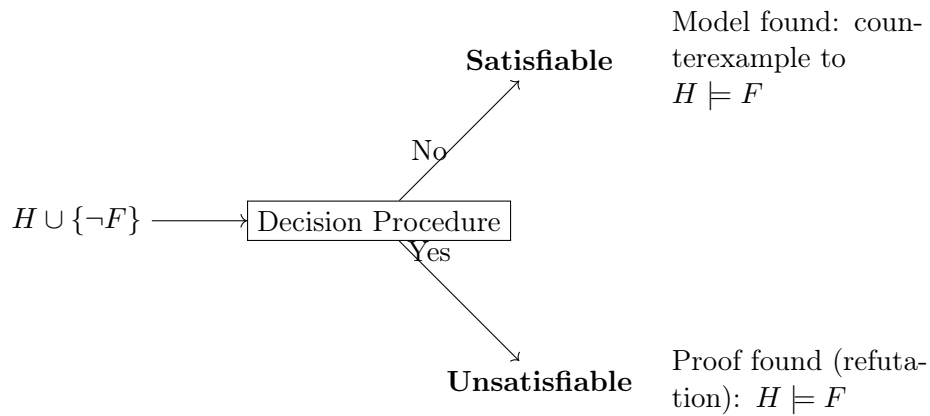
Refutational reasoning (or **proof by refutation**) is a fundamental technique in automated reasoning based on the principle of *reductio ad absurdum*. To prove that a conclusion F follows from hypotheses H , we assume the negation of the conclusion and derive a contradiction.

The method proceeds as follows:

1. **Goal:** Prove $H \models F$ (where H represents assumptions and F represents the conjecture)
2. **Method:** Show that $H \cup \{\neg F\}$ is unsatisfiable
3. **Conclusion:** If $H \cup \{\neg F\}$ is unsatisfiable, then $H \models F$ by the previous theorem

10.2 The Refutational Framework

The automated reasoning process can be formalized as follows:



Two possible outcomes:

- **Unsatisfiable:** The procedure derives a contradiction, producing a **refutation** (proof) that $H \models F$
- **Satisfiable:** The procedure finds a model $\mathcal{I}$ of $H \cup \{\neg F\}$, which serves as a **counterexample** showing that $H \not\models F$

This refutational approach forms the foundation of modern automated theorem provers and SAT solvers, reducing the problem of proving logical consequence to the problem of detecting unsatisfiability.

Part III

Free Variables and Satisfiability

11 Satisfiability of Formulas with Free Variables

11.1 The Problem of Free Variables

In our previous definition of satisfiability, we considered the semantics of logical connectives and quantifiers, but we did not fully address the treatment of **free variables**. Recall that a variable can be either:

- **Bound:** The variable falls within the scope of a quantifier
- **Free:** The variable does not fall within the scope of any quantifier

The presence of free variables requires careful consideration when determining satisfiability, as their interpretation depends on the variable assignment β .

11.2 Motivating Example

Consider the following formula:

$$F : \forall x (f(x, x) = x \wedge R(x, y))$$

This formula contains:

- A bound variable x (quantified by $\forall$)
- A free variable y (not quantified)

11.3 Semantic Evaluation with Free Variables

To determine whether an interpretation $\mathcal{I} = \langle D, \Phi \rangle$ with assignment β satisfies F , we write $\mathcal{I} \models_{\beta} F$ and proceed as follows:

By the semantics of universal quantification, we need to check that for all $d \in D$:

$$\mathcal{I} \models_{\beta[x \leftarrow d]} (f(x, x) = x \wedge R(x, y))$$

This requires verifying both conjuncts under the updated assignment $\beta[x \leftarrow d]$:

First conjunct: $\mathcal{I} \models_{\beta[x \leftarrow d]} f(x, x) = x$

This holds if and only if:

$$(\Phi(f)(d, d), d) \in \Phi(=)$$

where $\Phi(=)$ represents the identity relation. In other words, we require that $\Phi(f)(d, d) = d$ for all $d \in D$.

Second conjunct: $\mathcal{I} \models_{\beta[x \leftarrow d]} R(x, y)$

This holds if and only if:

$$(d, \beta(y)) \in \Phi(R)$$

Let $\beta(y) = d'$ for some $d' \in D$. Then we require $(d, d') \in \Phi(R)$ for all $d \in D$.

11.4 Existential Closure

The key observation is that the satisfiability of a formula with free variables is intimately connected to its **existential closure**.

Existential Closure

Definition 11.1 (Existential Closure). The existential closure of a formula F , denoted $\exists^* F$, is obtained by adding an existential quantifier for every free variable in F .

For example, if F contains free variables $y_1, y_2, \dots, y_n$, then:

$$\exists^* F = \exists y_1 \exists y_2 \cdots \exists y_n F$$

Returning to our example, if we consider the existential closure by quantifying the free variable y :

$$\exists y \forall x (f(x, x) = x \wedge R(x, y))$$

Under this interpretation, we need to verify that there exists some $d' \in D$ such that for all $d \in D$:

$$\mathcal{I} \models_{\beta[x \leftarrow d, y \leftarrow d']} R(x, y)$$

which means $(d, d') \in \Phi(R)$ for all $d \in D$.

11.5 Satisfiability for Open Formulas

We now provide the complete definition of satisfiability that handles both closed and open formulas.

Satisfiability - Complete Definition

Definition 11.2 (Satisfiability - Complete Definition). A formula F is **satisfiable** if:

- **Case 1 (Closed formula)**: If F is a sentence (contains no free variables), then F is satisfiable if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.
- **Case 2 (Open formula)**: If F contains free variables, then F is satisfiable if there exists an interpretation $\mathcal{I}$ and a variable assignment β such that $\mathcal{I} \models_{\beta} F$.

Theorem 11.1 (Satisfiability and Existential Closure). *A formula F with free variables is satisfiable if and only if its existential closure $\exists^* F$ is satisfiable.*

Proof. Let F be a formula with free variables $y_1, \dots, y_n$, and let $\exists^* F = \exists y_1 \cdots \exists y_n F$.

($\Rightarrow$) Suppose F is satisfiable. Then there exists an interpretation $\mathcal{I}$ and assignment β such that $\mathcal{I} \models_{\beta} F$. Let $d_i = \beta(y_i)$ for $i = 1, \dots, n$. Then by the semantics of existential quantification applied repeatedly, $\mathcal{I} \models \exists^* F$.

($\Leftarrow$) Suppose $\exists^* F$ is satisfiable. Then there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \exists y_1 \cdots \exists y_n F$. By the semantics of existential quantification, there exist $d_1, \dots, d_n \in D$ such that $\mathcal{I} \models_{\beta[y_1 \leftarrow d_1, \dots, y_n \leftarrow d_n]} F$. Therefore, F is satisfiable. $\square$

This result demonstrates that when checking satisfiability of formulas with free variables, we can equivalently check the satisfiability of their existential closures, treating free variables as existentially quantified.

11.6 Universal Closure and Validity

Dually, we can consider the **universal closure** of a formula with free variables.

Theorem 11.2 (Validity and Universal Closure). *A formula F with free variables is valid if and only if its universal closure $\forall^* F$ is valid.*

The intuition is straightforward: if F contains free variables, then F is valid if for all interpretations $\mathcal{I}$ and for all assignments β , we have $\mathcal{I} \models_\beta F$. This is precisely the condition for the universal closure $\forall^* F$ to be valid.

12 Decidability and Semidecidability in First-Order Logic

12.1 Computational Complexity of First-Order Logic

While certain restricted fragments of first-order logic admit decision procedures, the general case presents fundamental computational limitations:

Decidability in First-Order Logic

- **Unsatisfiability** in first-order logic is **semidecidable** (recursively enumerable)
- **Satisfiability** in first-order logic is **not even semidecidable**

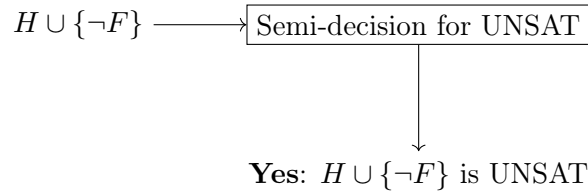
Note. Recall that if a decision problem A is semidecidable and its complement $\neg A$ is also semidecidable, then A is decidable. The decision procedure can be obtained by running both semidecision procedures in parallel: if the instance belongs to A , the first procedure will eventually terminate with "yes"; if the instance belongs to $\neg A$, the second procedure will eventually terminate with "yes".

12.2 Review: Semi-Decision Procedures

Consider the problem of determining whether $H \models F$ (logical consequence).

Semi-decision procedure for unsatisfiability:

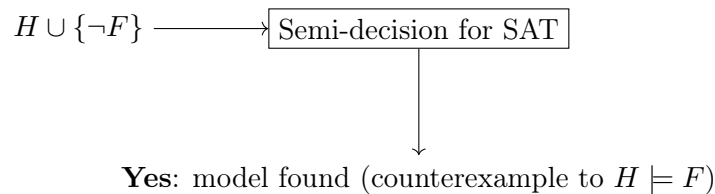
? (may not terminate)



If the procedure terminates with "yes", we have proven $H \models F$ by refutation. However, if $H \cup \{\neg F\}$ is satisfiable, the procedure may run indefinitely.

Semi-decision procedure for satisfiability:

? (may not terminate)



If this procedure terminates with "yes", we have found a countermodel showing $H \not\models F$. However, such a procedure does not exist for general first-order logic.

12.3 The Need for Restricted Theories

To obtain decision procedures, we must restrict our attention to **decidable fragments** of first-order logic. This motivates the study of **first-order theories**—logical systems with additional structure that enable algorithmic reasoning.

13 First-Order Theories

13.1 Definition of a First-Order Theory

First-Order Theory

Definition 13.1 (First-Order Theory). A **first-order theory** is a pair $\mathcal{T} = \langle \Sigma, \mathcal{A} \rangle$ where:

- Σ is a **signature** (defining the vocabulary of the theory)
- $\mathcal{A}$ is a set of **axioms**—closed formulas (sentences) that we assume to hold in all models of the theory

The axioms define properties of specific symbols in the signature, giving them fixed interpretations across all models of the theory.

13.2 The Theory of Equality

A fundamental example is the **theory of equality** $\mathcal{T}_E = \langle \Sigma_E, \mathcal{A}_E \rangle$.

Signature: The signature of the theory of equality is:

$$\Sigma_E = \{=\} \cup C \cup \mathcal{F} \cup \mathcal{P}$$

where:

- $=$ is the equality predicate (interpreted symbol)
- C is a set of constant symbols (uninterpreted)
- $\mathcal{F}$ is a set of function symbols (uninterpreted)
- $\mathcal{P}$ is a set of predicate symbols (uninterpreted)

Example signature: Consider $\Sigma = \{a, b, f, Q, =\}$. The symbols a, b, f, Q are **free** (uninterpreted), whereas $=$ (equality) is **defined** by the axioms of the theory.

13.3 Axioms of Equality

The axiom set $\mathcal{A}_E$ consists of formulas that characterize equality as a congruence relation.

Equivalence relation axioms:

$$\begin{array}{ll} \forall x (x = x) & \text{(Reflexivity)} \\ \forall x \forall y (x = y \rightarrow y = x) & \text{(Symmetry)} \\ \forall x \forall y \forall z ((x = y \wedge y = z) \rightarrow x = z) & \text{(Transitivity)} \end{array}$$

These three axioms establish that equality is an equivalence relation.

Congruence axioms for function symbols: For each function symbol $f \in \mathcal{F}$ of arity n :

$$\forall \vec{x} \forall \vec{y} \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow f(x_1, \dots, x_n) = f(y_1, \dots, y_n)$$

where $\vec{x} = (x_1, \dots, x_n)$ and $\vec{y} = (y_1, \dots, y_n)$.

This axiom states that f respects equality: equal inputs produce equal outputs.

Congruence axioms for predicate symbols: For each predicate symbol $R \in \mathcal{P}$ of arity n :

$$\forall \vec{x} \forall \vec{y} \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow (R(x_1, \dots, x_n) \leftrightarrow R(y_1, \dots, y_n))$$

This axiom states that R respects equality: the truth value of R depends only on the equivalence classes of its arguments.

13.4 Models of a Theory

Definition 13.2 ($\mathcal{T}$ -Model). A $\mathcal{T}$ -model is an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \mathcal{A}$ (i.e., $\mathcal{I}$ satisfies all axioms in $\mathcal{A}$).

13.5 Satisfiability and Validity Relative to a Theory

Definition 13.3 ($\mathcal{T}$ -Satisfiability). A formula G is $\mathcal{T}$ -satisfiable if there exists a $\mathcal{T}$ -model $\mathcal{I}$ such that $\mathcal{I} \models G$.

Definition 13.4 ($\mathcal{T}$ -Validity). A formula G is $\mathcal{T}$ -valid if for all $\mathcal{T}$ -models $\mathcal{I}$, we have $\mathcal{I} \models G$.

Equivalently, G is a logical consequence of the axioms: $\mathcal{A} \models G$.

13.6 Example: Satisfiability vs. $\mathcal{T}_E$ -Satisfiability

Consider the set of formulas:

$$S = \{P(a), \neg P(b), a = b\}$$

Claim: S is satisfiable in general first-order logic, but S is **not** $\mathcal{T}_E$ -satisfiable.

Proof. Satisfiability: We can construct an interpretation $\mathcal{I}$ with domain $D = \{d_1, d_2\}$ where:

- $\Phi(a) = d_1, \Phi(b) = d_2$
- $\Phi(P) = \{d_1\}$
- $\Phi(=) = \{(d_1, d_1), (d_2, d_2), (d_1, d_2), (d_2, d_1)\}$ (universal relation)

Then $\mathcal{I} \models P(a)$, $\mathcal{I} \models \neg P(b)$, and $\mathcal{I} \models a = b$, so $\mathcal{I} \models S$.

$\mathcal{T}_E$ -Unsatisfiability: In any $\mathcal{T}_E$ -model, equality must satisfy the congruence axioms. If $\mathcal{I} \models a = b$ and $\mathcal{I} \models P(a)$, then by the congruence axiom for P , we must have $\mathcal{I} \models P(b)$, contradicting $\mathcal{I} \models \neg P(b)$. Therefore, no $\mathcal{T}_E$ -model satisfies S . $\square$

This example illustrates that satisfiability relative to a theory is more restrictive than general satisfiability, as models must respect the additional constraints imposed by the theory's axioms.

14 Quantifier-Free Fragments and Decidability

14.1 The Quantifier-Free Fragment of $\mathcal{T}_E$

While the full theory of equality $\mathcal{T}_E$ is undecidable, its **quantifier-free fragment** admits a decision procedure.

Definition 14.1 (Quantifier-Free Fragment). The **quantifier-free fragment** of first-order logic consists of all formulas that do not contain quantifiers ($\forall$ or $\exists$).

For a theory $\mathcal{T}$, the quantifier-free fragment allows quantifiers to appear only in the axioms, not in the input formulas to be checked for satisfiability.

Our goal is to develop a decision procedure for the quantifier-free fragment of $\mathcal{T}_E$. We will focus initially on deciding satisfiability for sets (or conjunctions) of literals, and then extend this to arbitrary quantifier-free formulas using normal forms.

14.2 Normal Forms

To handle arbitrary quantifier-free formulas, we introduce two important normal forms that facilitate systematic reasoning.

14.2.1 Negation Normal Form (NNF)

Negation Normal Form (NNF)

Definition 14.2 (Negation Normal Form). A formula is in **Negation Normal Form (NNF)** if:

- The only logical connectives are $\wedge$, $\vee$, and $\neg$
- Negation ($\neg$) applies only to atomic formulas (atoms)

Any quantifier-free formula can be converted to NNF by applying the following equivalence-preserving transformations:

$$\begin{array}{ll}
 H \leftrightarrow G \equiv (H \rightarrow G) \wedge (G \rightarrow H) & \text{(Eliminate biconditional)} \\
 H \rightarrow G \equiv \neg H \vee G & \text{(Eliminate implication)} \\
 \neg(H \wedge G) \equiv \neg H \vee \neg G & \text{(De Morgan's law)} \\
 \neg(H \vee G) \equiv \neg H \wedge \neg G & \text{(De Morgan's law)} \\
 \neg\neg H \equiv H & \text{(Double negation elimination)}
 \end{array}$$

14.2.2 Disjunctive Normal Form (DNF)

Disjunctive Normal Form (DNF)

Definition 14.3 (Disjunctive Normal Form). A formula is in **Disjunctive Normal Form (DNF)** if it is a disjunction of conjunctions of literals:

$$\bigvee_{i=1}^k \left(\bigwedge_{j=1}^{n_i} L_{i,j} \right)$$

where each $L_{i,j}$ is a literal (an atomic formula or its negation).

Equivalently, a DNF formula has the structure:

$$(L_{1,1} \wedge L_{1,2} \wedge \cdots \wedge L_{1,n_1}) \vee (L_{2,1} \wedge L_{2,2} \wedge \cdots \wedge L_{2,n_2}) \vee \cdots \vee (L_{k,1} \wedge L_{k,2} \wedge \cdots \wedge L_{k,n_k})$$

14.3 Conversion to DNF

Any quantifier-free formula F can be systematically converted to an equivalent DNF formula through a two-stage process:

$$F \longrightarrow \boxed{\text{Convert to NNF}} \longrightarrow F' \longrightarrow \boxed{\text{Convert to DNF}} \longrightarrow F''$$

Stage 1: Convert F to NNF using the transformations above, obtaining F' .

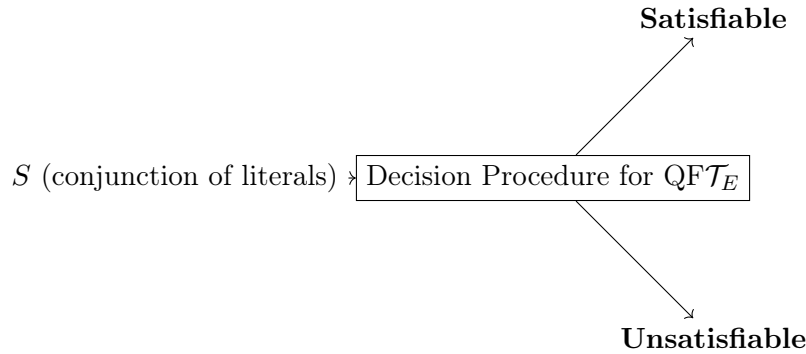
Stage 2: Convert F' to DNF by repeatedly applying the distributivity law:

$$H \wedge (G_1 \vee G_2) \equiv (H \wedge G_1) \vee (H \wedge G_2)$$

This transformation pushes conjunctions inward and disjunctions outward, resulting in a disjunction of conjunctions.

14.4 Decision Procedure for Quantifier-Free $\mathcal{T}_E$

The decision procedure operates on conjunctions of literals. Given a set (or conjunction) S of literals:



Extension to arbitrary quantifier-free formulas: For a general quantifier-free formula F :

1. Convert F to NNF, obtaining F'
2. Convert F' to DNF, obtaining $F'' = \bigvee_{i=1}^k C_i$ where each C_i is a conjunction of literals
3. Apply the decision procedure to each conjunction C_i
4. F is satisfiable if and only if at least one C_i is satisfiable

This reduction allows us to decide satisfiability for arbitrary quantifier-free formulas by deciding satisfiability for each disjunct independently.

14.5 Elimination of Non-Equality Predicates

The decision procedure for quantifier-free $\mathcal{T}_E$ assumes that the only predicate symbol in the formula is equality ($=$). However, formulas may contain additional predicate symbols. We now show how to eliminate such predicates through a satisfiability-preserving transformation.

14.5.1 The Transformation

Predicate Elimination

Definition 14.4 (Predicate Elimination). Given an occurrence of a predicate symbol P of arity n applied to terms $t_1, \dots, t_n$, we eliminate P by introducing:

- A fresh function symbol f_P (one for each predicate symbol $P \neq =$)
- A fresh constant symbol $\bullet$ (the same constant for all predicates)

The transformation replaces predicate applications with equality constraints:

$$\begin{aligned} P(t_1, \dots, t_n) &\mapsto f_P(t_1, \dots, t_n) = \bullet \\ \neg P(t_1, \dots, t_n) &\mapsto f_P(t_1, \dots, t_n) \neq \bullet \end{aligned}$$

Intuition: The predicate $P(t_1, \dots, t_n)$ is true if and only if the term $f_P(t_1, \dots, t_n)$ evaluates to the distinguished constant $\bullet$.

14.5.2 Satisfiability Preservation

Theorem 14.1 (Satisfiability Preservation). *Let F be a quantifier-free formula and let F' be the result of applying predicate elimination to F . Then F is satisfiable if and only if F' is satisfiable in models with cardinality greater than 1.*

Note. The restriction to models with cardinality greater than 1 is necessary to avoid trivial models. In a model with domain $D = \{d\}$ (cardinality 1), all terms evaluate to the same element d , making all equalities trivially true and potentially invalidating the transformation.

14.5.3 Example: The Need for Cardinality Restriction

Consider the following two formulas:

Original formula:

$$S_1 = \{\forall x \forall y (x = y), \neg P(a)\}$$

This formula is $\mathcal{T}_E$ -satisfiable. We can construct a model with domain $D = \{d_1, d_2\}$ where:

- $\Phi(a) = d_1$
- $\Phi(P) = \{d_2\}$ (so $d_1 \notin \Phi(P)$, making $\neg P(a)$ true)
- The first axiom $\forall x \forall y (x = y)$ is vacuously false in this model, but if we interpret it as requiring equality to be universal, we need a different construction

Actually, let's reconsider: $\{\forall x \forall y (x = y), \neg P(a)\}$ asserts that all elements are equal (domain has cardinality 1) and $P(a)$ is false. This is satisfiable in a model with $D = \{d\}$, $\Phi(a) = d$, and $\Phi(P) = \emptyset$.

Transformed formula:

$$S_2 = \{\forall x \forall y (x = y), f_P(a) \neq \bullet\}$$

In a model with cardinality 1, we have $D = \{d\}$, so $\Phi(f_P)(d) = d$ and $\Phi(\bullet) = d$. Therefore, $f_P(a) = \bullet$, contradicting $f_P(a) \neq \bullet$. Thus S_2 is unsatisfiable in models of cardinality 1.

This example demonstrates that the transformation preserves satisfiability only when we exclude trivial models with cardinality 1.

14.6 Treatment of Free Variables

In quantifier-free formulas, free variables may appear. For the purposes of satisfiability checking, free variables are treated analogously to constants.

Note (Free Variables as Constants). To determine satisfiability of a formula F with free variables $x_1, \dots, x_k$, we need to find an interpretation $\mathcal{I}$ and an assignment β such that $\mathcal{I} \models_{\beta} F$.

Equivalently, we can treat each free variable x_i as a fresh constant symbol c_i and check satisfiability of the resulting closed formula. The formula is satisfiable if and only if there exist domain elements that can be assigned to these constants to satisfy the formula.

This observation allows us to reduce the satisfiability problem for formulas with free variables to the satisfiability problem for closed formulas by a simple syntactic substitution.

15 The Congruence Closure Algorithm

15.1 Motivating Example

Consider the following conjunction of literals:

$$x = g(x) \wedge f(x, g(g(x))) \neq g(x) \wedge f(x, x) = g(g(x))$$

Question: Is this formula satisfiable or unsatisfiable?

Intuitive analysis: This formula appears to be unsatisfiable. From $x = g(x)$, we can derive $g(x) = g(g(x))$ by applying the function g to both sides (congruence). Then from $f(x, x) = g(g(x))$ and the derived equality $g(x) = g(g(x))$, we obtain $f(x, x) = g(x)$. But we also have $x = g(x)$, so by congruence on f , we get $f(x, x) = f(x, g(x))$. However, we need to check whether $f(x, g(g(x))) = g(x)$ follows from the positive equalities, which would contradict $f(x, g(g(x))) \neq g(x)$.

15.2 The Congruence Closure Decision Procedure

Congruence Closure Algorithm

The **congruence closure algorithm** is a decision procedure for satisfiability in the quantifier-free fragment of the theory of equality. The algorithm operates as follows:

1. **Build the smallest congruence:** Construct the smallest congruence relation that satisfies all positive equalities in the input formula
2. **Check negative equalities:** For each negative equality $s \neq t$ in the input, check whether s and t belong to the same equivalence class in the congruence
3. **Determine satisfiability:**
 - If any pair (s, t) with $s \neq t$ belongs to the same equivalence class, the formula is **unsatisfiable**
 - Otherwise, the congruence provides a model, and the formula is **satisfiable**

The key insight is that the smallest congruence containing all positive equalities is included in every congruence that satisfies those equalities. Therefore, if a negative equality is violated in the smallest congruence, it must be violated in every model.

15.3 Equivalence Relations and Partitions

Before describing the algorithm in detail, we recall fundamental properties of equivalence relations.

Equivalence Relation

Definition 15.1 (Equivalence Relation). An **equivalence relation** R on a set S is a binary relation that is:

- **Reflexive:** $\forall s \in S, s R s$
- **Symmetric:** $\forall s, t \in S, s R t \implies t R s$
- **Transitive:** $\forall s, t, u \in S, (s R t \wedge t R u) \implies s R u$

Definition 15.2 (Equivalence Class). Given an equivalence relation R on set S , the **equivalence class** of an element $s \in S$ is:

$$[s]_R = \{s' \in S \mid s' R s\}$$

Definition 15.3 (Quotient Set). The **quotient set** S/R is the set of all equivalence classes:

$$S/R = \{[s]_R \mid s \in S\}$$

Proposition 15.1 (Partition Property). *An equivalence relation R on S induces a partition of S . That is, for any two distinct equivalence classes $[s]_R$ and $[p]_R$:*

$$s \not\sim_R p \implies [s]_R \cap [p]_R = \emptyset$$

where $s \sim_R p$ denotes $s R p$.

Conversely, every element belongs to exactly one equivalence class, and the union of all equivalence classes equals S .

This partition property is fundamental to the congruence closure algorithm, as it allows us to represent the congruence relation efficiently using a union-find data structure.

Part IV

Decision Procedures for Quantifier-Free Theories

16 The Congruence Closure Algorithm

16.1 Computing Congruence Closure

We now describe how to compute the congruence closure for the quantifier-free fragment of the theory of equality $\mathcal{T}_E$.

16.1.1 Input Problem Structure

Consider an input formula F consisting of a conjunction of equality and disequality literals:

$$F = \underbrace{\{s_1 = t_1, \dots, s_n = t_n\}}_{F^+} \cup \underbrace{\{s_{n+1} \neq t_{n+1}, \dots, s_{n+m} \neq t_{n+m}\}}_{F^-}$$

where:

- F^+ denotes the set of **positive equalities** (equations)
- F^- denotes the set of **negative equalities** (disequations)

The congruence closure algorithm will construct the smallest congruence containing all equalities in F^+ , then check whether any disequality in F^- is violated.

16.1.2 Equivalence Relations and Representatives

Given a binary equivalence relation $R \subseteq S \times S$ on a set S , we can construct its quotient:

$$S/R = \{[s]_R \mid s \in S\}$$

where the equivalence class of s is:

$$[s]_R = \{p \in S \mid p R s\}$$

The element s is called a **representative** of the equivalence class $[s]_R$.

16.1.3 Disjointness of Equivalence Classes

Proposition 16.1 (Disjoint Classes). *For any two equivalence classes $[s_1]_R$ and $[s_2]_R$, either they are identical or they are disjoint:*

$$[s_1]_R \cap [s_2]_R \neq \emptyset \implies [s_1]_R = [s_2]_R$$

Proof. Suppose there exists $p \in [s_1]_R \cap [s_2]_R$. Then:

- $p R s_1$ (since $p \in [s_1]_R$)
- $p R s_2$ (since $p \in [s_2]_R$)

By symmetry and transitivity of R , we have $s_1 R s_2$. Therefore, for any $q \in [s_1]_R$, we have $q R s_1$ and $s_1 R s_2$, so by transitivity $q R s_2$, which means $q \in [s_2]_R$. Similarly, $[s_2]_R \subseteq [s_1]_R$. Thus $[s_1]_R = [s_2]_R$. $\square$

16.1.4 Example: Equivalence Closure

Consider the binary relation:

$$R = \{(a, b), (b, c), (d, d)\}$$

We can visualize this as a directed graph:

$$a \longrightarrow b \longrightarrow c \qquad d \curvearrowright$$

The **equivalence closure** R^* of R must include:

- **Reflexivity:** $(a, a), (b, b), (c, c), (d, d)$
- **Symmetry:** $(b, a), (c, b)$
- **Transitivity:** $(a, c), (c, a)$

Thus:

$$R^* = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c), (c, a), (c, b), (c, c), (d, d)\}$$

The quotient set is:

$$S/R^* = \{\{a, b, c\}, \{d\}\}$$

Equivalence Closure

Definition 16.1 (Equivalence Closure). Given a binary relation R on set S , the **equivalence closure** of R , denoted R^* , is the $\subseteq$ -smallest equivalence relation such that:

1. R^* is an equivalence relation
2. $R \subseteq R^*$

Equivalently, R^* is the intersection of all equivalence relations containing R .

The equivalence closure can be computed by iteratively applying reflexivity, symmetry, and transitivity rules until a fixed point is reached.

16.1.5 Understanding the $\subseteq$ -Smallest Property

Proposition 16.2 (Minimality of Equivalence Closure). *Let R be a binary relation and R^* be its equivalence closure. For any equivalence relation P such that $R \subseteq P$, we have $R^* \subseteq P$.*

This means that R^* is the **smallest** equivalence relation containing R with respect to the subset ordering $\subseteq$.

Congruence Closure

Definition 16.2 (Congruence Closure). Given a binary relation R , the **congruence closure** of R , denoted R_{cong}^* , is the $\subseteq$ -smallest relation such that:

1. R_{cong}^* is a congruence relation
2. $R \subseteq R_{\text{cong}}^*$

A congruence relation is an equivalence relation that is preserved under function application: if $s_1 \sim t_1, \dots, s_n \sim t_n$, then $f(s_1, \dots, s_n) \sim f(t_1, \dots, t_n)$ for all function symbols f .

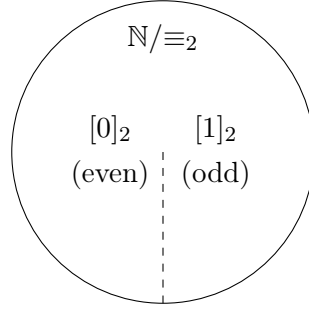
16.1.6 Example: Refinement of Equivalence Relations

To illustrate the concept of one equivalence relation being smaller (finer) than another, consider the following example on the natural numbers $\mathbb{N}$.

Equivalence modulo 2: Define the relation $\equiv_2 \subseteq \mathbb{N} \times \mathbb{N}$ by:

$$n \equiv_2 m \quad \text{iff} \quad n \equiv m \pmod{2}$$

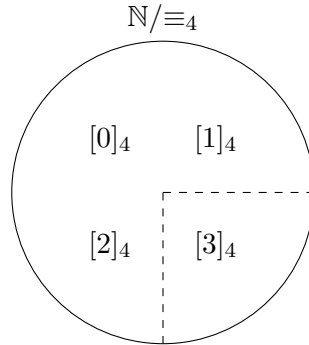
This relation partitions $\mathbb{N}$ into two equivalence classes:



Equivalence modulo 4: Define the relation $\equiv_4 \subseteq \mathbb{N} \times \mathbb{N}$ by:

$$n \equiv_4 m \quad \text{iff} \quad n \equiv m \pmod{4}$$

This relation partitions $\mathbb{N}$ into four equivalence classes:



Proposition 16.3 (Refinement). *The relation $\equiv_4$ is a **refinement** of $\equiv_2$. That is:*

$$n \equiv_4 m \quad \implies \quad n \equiv_2 m$$

However, the converse does not hold:

$$n \equiv_2 m \quad \not\implies \quad n \equiv_4 m$$

Proof. If $n \equiv_4 m$, then $n \equiv m \pmod{4}$, which means $n - m = 4k$ for some $k \in \mathbb{Z}$. Therefore, $n - m = 2(2k)$, so $n \equiv m \pmod{2}$, i.e., $n \equiv_2 m$.

For the converse, consider $n = 0$ and $m = 2$. Then $n \equiv_2 m$ (both are even), but $n \not\equiv_4 m$ (since $0 \equiv 0 \pmod{4}$ and $2 \equiv 2 \pmod{4}$). $\square$

Set-theoretic interpretation: This means that $(n, m) \in \equiv_4 \implies (n, m) \in \equiv_2$, so:

$$\equiv_4 \subseteq \equiv_2$$

In this sense, $\equiv_4$ is **strictly smaller** than $\equiv_2$ (it is a proper subset), and provides a finer partition of $\mathbb{N}$.

16.2 The Congruence Closure Decision Procedure

We now return to the main problem: deciding satisfiability for a quantifier-free formula in the theory of equality.

16.2.1 Algorithm Description

Recall that the input formula F has the form:

$$F = \underbrace{\{s_1 = t_1, \dots, s_n = t_n\}}_{F^+} \cup \underbrace{\{s_{n+1} \neq t_{n+1}, \dots, s_{n+m} \neq t_{n+m}\}}_{F^-}$$

The congruence closure algorithm proceeds as follows:

Algorithm 1 Congruence Closure Decision Procedure

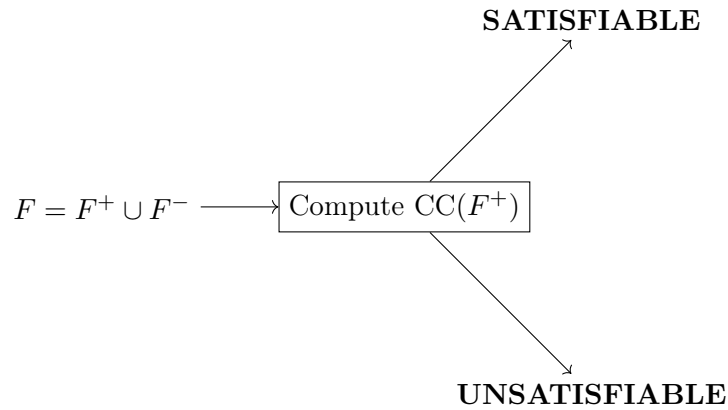
- 1: **Input:** Formula $F = F^+ \cup F^-$
 - 2: Compute the congruence closure $\text{CC}(F^+)$ of the positive equalities F^+
 - 3: **for** $i = n + 1$ to $n + m$ **do**
 - 4: **if** $(s_i, t_i) \in \text{CC}(F^+)$ **then**
 - 5: **return** UNSATISFIABLE
 - 6: **end if**
 - 7: **end for**
 - 8: **return** SATISFIABLE
-

Step 1: Compute $\text{CC}(F^+)$, the smallest congruence relation containing all pairs (s_i, t_i) for $i = 1, \dots, n$.

Step 2: For each disequality $s_i \neq t_i$ in F^- (where $n + 1 \leq i \leq n + m$), check whether (s_i, t_i) belongs to the same equivalence class in $\text{CC}(F^+)$.

Step 3:

- If $(s_j, t_j) \in \text{CC}(F^+)$ for some j with $n + 1 \leq j \leq n + m$, then F is **unsatisfiable**
- Otherwise, F is **satisfiable**



16.2.2 Correctness of the Algorithm

Theorem 16.4 (Soundness and Completeness). *The congruence closure algorithm is sound and complete for deciding satisfiability in the quantifier-free fragment of $\mathcal{T}_E$.*

Proof sketch. Soundness (Unsatisfiability): Suppose there exists j with $n + 1 \leq j \leq n + m$ such that $(s_j, t_j) \in \text{CC}(F^+)$.

Since $\text{CC}(F^+)$ is the smallest congruence relation containing F^+ , the pair (s_j, t_j) must belong to *every* congruence relation that contains F^+ .

Therefore, in any model $\mathcal{I}$ that satisfies all equalities in F^+ , we must have $\mathcal{I} \models s_j = t_j$. But F also contains $s_j \neq t_j$, so $\mathcal{I} \not\models s_j \neq t_j$. Thus, no model can satisfy both F^+ and F^- , making F unsatisfiable.

Completeness (Satisfiability): If $(s_i, t_i) \notin \text{CC}(F^+)$ for all i with $n + 1 \leq i \leq n + m$, then we can construct a model by interpreting the equivalence classes of $\text{CC}(F^+)$ as domain elements. This model satisfies all equalities in F^+ (by construction) and all disequalities in F^- (since the terms belong to different equivalence classes). $\square$

The key insight is that the congruence closure captures exactly those equalities that are *forced* by the positive equalities in F^+ together with the congruence axioms of equality.

16.3 Examples: Manual Execution

We now illustrate the congruence closure algorithm with concrete examples.

16.3.1 Notation and Setup

For a formula F of the form:

$$F = \{s_1 = t_1, \dots, s_n = t_n\} \cup \{s_{n+1} \neq t_{n+1}, \dots, s_{n+m} \neq t_{n+m}\}$$

we define S_F as the **set of all terms** that appear in F .

16.3.2 Example 1

Consider the formula:

$$F = \{x = g(x), f(x, x) = g(g(x)), f(x, g(g(x))) \neq g(x)\}$$

The set of terms is:

$$S_F = \{x, g(x), f(x, x), g(g(x)), f(x, g(g(x)))\}$$

Notation:

- x is a **free variable symbol**
- f is a **binary function symbol** (arity 2)
- g is a **unary function symbol** (arity 1)

Positive equalities: $F^+ = \{x = g(x), f(x, x) = g(g(x))\}$

Negative equalities: $F^- = \{f(x, g(g(x))) \neq g(x)\}$

16.3.3 Example 2

Consider the formula:

$$F = \{f(a, b) = a, f(f(a, b), b) \neq a\}$$

The set of terms is:

$$S_F = \{f(a, b), a, b, f(f(a, b), b)\}$$

Notation:

- a, b are **constant symbols** (by convention)
- f is a **binary function symbol**

Positive equalities: $F^+ = \{f(a, b) = a\}$

Negative equalities: $F^- = \{f(f(a, b), b) \neq a\}$

Note (Free Variables vs. Constants). In the quantifier-free fragment, all variables are free. From a semantic perspective, free variables and constants can be used interchangeably: both are interpreted as arbitrary elements of the domain. The distinction is purely syntactic.

16.3.4 Step-by-Step Execution for Example 1

We now execute the congruence closure algorithm on Example 1:

$$F = \{x = g(x), f(x, x) = g(g(x)), f(x, g(g(x))) \neq g(x)\}$$

Step 1: Initialize the partition

Create a partition of S_F where every term is in its own singleton equivalence class:

$$\{\{x\}, \{g(x)\}, \{f(x, x)\}, \{g(g(x))\}, \{f(x, g(g(x)))\}\}$$

Step 2: Process positive equalities

For each equality $(s_i = t_i) \in F^+$ (for all i with $1 \leq i \leq n$):

- Unite the equivalence class of s_i with the equivalence class of t_i
- Unite the classes of all terms in S_F that become congruent as a consequence of $s_i = t_i$

Processing $x = g(x)$:

Merge the classes $\{x\}$ and $\{g(x)\}$:

$$\{\{x, g(x)\}, \{f(x, x)\}, \{g(g(x))\}, \{f(x, g(g(x)))\}\}$$

Since $x = g(x)$, by congruence we derive $g(x) = g(g(x))$ (applying g to both sides). Merge these classes:

$$\{\{x, g(x), g(g(x))\}, \{f(x, x)\}, \{f(x, g(g(x)))\}\}$$

Processing $f(x, x) = g(g(x))$:

Since $g(g(x)) \in \{x, g(x), g(g(x))\}$ and $f(x, x) \in \{f(x, x)\}$, merge these classes:

$$\{\{x, g(x), g(g(x)), f(x, x)\}, \{f(x, g(g(x)))\}\}$$

Now check for congruence: since $x \sim g(g(x))$ (both in the same class), we have:

$$f(x, x) \sim f(x, g(g(x)))$$

by the congruence property (both arguments are in the same equivalence classes). Merge these classes:

$$\{\{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}\}$$

Step 3: Check negative equalities

Check whether $(s_i, t_i) \in \text{CC}(F^+)$ for all i with $n + 1 \leq i \leq n + m$.

For the disequality $f(x, g(g(x))) \neq g(x)$:

- $f(x, g(g(x))) \in \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$
- $g(x) \in \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$

Both terms are in the same equivalence class, so $(f(x, g(g(x))), g(x)) \in \text{CC}(F^+)$.

Conclusion: The formula F is **UNSATISFIABLE**.

17 Implementation of Congruence Closure on a DAG

17.1 DAG Representation

The congruence closure algorithm can be implemented efficiently using a **Directed Acyclic Graph (DAG)** representation of the terms.

Consider the formula:

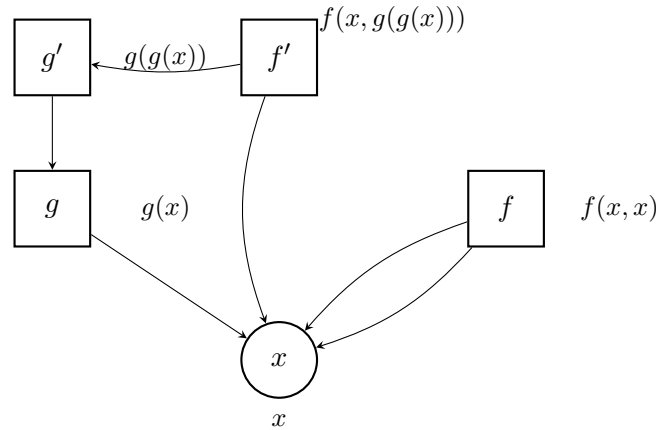
$$F = \{x = g(x), f(x, x) = g(g(x))\} \cup \{f(x, g(g(x))) \neq g(x)\}$$

The set of terms is:

$$S_F = \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$$

We represent these terms as a graph where:

- Each term is a node
- Edges connect function symbols to their arguments
- We use g and g' to distinguish the two applications of function g
- We use f and f' to distinguish the two applications of function f



The graph shows the structure of the terms:

- Node g has an edge to x (representing $g(x)$)
- Node f has two edges to x (representing $f(x, x)$)
- Node g' has an edge to g (representing $g(g(x))$)
- Node f' has edges to x and g' (representing $f(x, g(g(x)))$)

This is a graph representation of the term structure, which allows efficient implementation of the congruence closure algorithm using union-find data structures to track equivalence classes of nodes.

17.2 Node Data Structure

Node Data Structure

Each node in the DAG is represented by a data structure with the following fields:

```
type node = {
  id: Id;
  fn: String;
  args: Id list;
  mutable find: Id;
  mutable ccpair: Id set;
}
```

For a node n of type `node`, the fields have the following meaning:

- **n.id**: The unique identifier of node n
- **n.fn**: The function symbol (or variable name) represented by this node
- **n.args**: The list of identifiers of the argument nodes (empty for variables and constants)
- **n.find**: The identifier of the representative node of the equivalence class to which n belongs
 - *Initialization*: **n.find** = **n.id** (each node starts in its own equivalence class)
- **n.ccpair**: The set of identifiers of the parent nodes of n (nodes that have n as an argument) and all nodes congruent to n

This data structure supports the union-find operations needed for the congruence closure algorithm, where `find` implements the path to the representative of each equivalence class, and `ccpair` tracks congruence relationships for efficient propagation.

17.3 Basic Operations

We now define the basic operations on the DAG structure.

17.3.1 The NODE Function

The function `NODE` retrieves a node given its identifier:

$$\text{NODE} : \text{Id} \rightarrow \text{Node}$$

$$\text{NODE}(i) = n \quad \text{such that} \quad n.\text{id} = i$$

17.3.2 The FIND Operation

The `FIND` function takes an identifier and returns the identifier of the representative of the equivalence class to which the node belongs:

$$\text{FIND} : \text{Id} \rightarrow \text{Id}$$

```
let rec FIND i =
  let n = NODE i in
  if n.find = i then i
  else FIND n.find
```

This recursive function follows the `find` pointers until it reaches a node that is its own representative (i.e., $n.\text{find} = n.\text{id}$).

17.3.3 The UNION Operation

The UNION function merges two equivalence classes:

UNION : $\text{Id} \times \text{Id} \rightarrow \text{void}$

```
let UNION i_1 i_2 =
  let n_1 = NODE (FIND i_1) in
  let n_2 = NODE (FIND i_2) in
  n_1.find <- n_2.find;
  n_2.ccpair <- n_2.ccpair U n_1.ccpair;
  n_1.ccpair <- empty_set
```

The UNION operation performs the following steps:

1. Find the representatives n_1 and n_2 of the two equivalence classes
2. Make n_1 point to n_2 (merging the classes)
3. Transfer the `ccpair` set from n_1 to n_2
4. Clear the `ccpair` set of n_1

Note. During execution, $n.\text{ccpair} \neq \emptyset$ if and only if n is a representative (i.e., $n.\text{find} = n.\text{id}$). This invariant ensures that only representative nodes maintain the parent and congruence information.

17.3.4 The CCPAR Function

The CCPAR function returns the `ccpair` set of the representative of a given node:

CCPAR : $\text{Id} \rightarrow \text{Id set}$

```
let CCPAR i = (NODE (FIND i)).ccpair
```

This function first finds the representative of node i , then returns its `ccpair` set.

17.3.5 The CONGRUENT Function

The CONGRUENT function checks whether two nodes are congruent (i.e., they have the same function symbol and their corresponding arguments are in the same equivalence classes):

CONGRUENT : $\text{Id} \times \text{Id} \rightarrow \text{bool}$

```
let CONGRUENT i_1 i_2 =
  let n_1 = NODE i_1 in
  let n_2 = NODE i_2 in
  n_1.fn = n_2.fn &&
  |n_1.args| = |n_2.args| &&
  forall i in {1, ..., |n_1.args|}
    FIND n_1.args[i] = FIND n_2.args[i]
```

Two nodes are congruent if:

1. They have the same function symbol ($n_1.\text{fn} = n_2.\text{fn}$)
2. They have the same number of arguments ($|n_1.\text{args}| = |n_2.\text{args}|$)

3. Their corresponding arguments are in the same equivalence classes

17.3.6 The MERGE Function

MERGE Operation

The **MERGE** function merges two equivalence classes and recursively propagates congruence:

```

MERGE : Id × Id → void

let rec MERGE i_1 i_2 =
  if FIND i_1 /= FIND i_2 then
    begin
      let P_1 = CCPAR i_1 in
      let P_2 = CCPAR i_2 in
      UNION i_1 i_2;
      foreach (t_1, t_2) in P_1 x P_2 do
        if FIND t_1 /= FIND t_2 && CONGRUENT t_1 t_2
          then MERGE t_1 t_2
      done
    end
  end

```

The **MERGE** operation works as follows:

1. Check if i_1 and i_2 are already in the same equivalence class; if so, do nothing
2. Save the **ccpar** sets P_1 and P_2 of both nodes
3. Perform **UNION** to merge the equivalence classes
4. For each pair (t_1, t_2) in the Cartesian product $P_1 \times P_2$:
 - If t_1 and t_2 are not already equivalent and are congruent
 - Recursively merge them with **MERGE**

This recursive propagation ensures that when two nodes become equivalent, all pairs of congruent parent nodes are also merged, maintaining the congruence closure property.

17.4 The Complete Congruence Closure Algorithm

We can now describe the complete congruence closure algorithm using the DAG-based implementation.

17.4.1 Algorithm

Complete Congruence Closure Algorithm (DAG-based)

Given a formula:

$$F = \{s_1 = t_1, \dots, s_n = t_n\} \cup \{s_{n+1} \neq t_{n+1}, \dots, s_{n+m} \neq t_{n+m}\}$$

The algorithm proceeds as follows:

Algorithm 2 Congruence Closure Algorithm (DAG-based)

```

1: Input: Formula  $F = F^+ \cup F^-$  where  $F^+ = \{s_1 = t_1, \dots, s_n = t_n\}$  and  $F^- = \{s_{n+1} \neq t_{n+1}, \dots, s_{n+m} \neq t_{n+m}\}$ 
2: Construct the DAG for  $F$ 
3: for  $i = 1$  to  $n$  do
4:    $\text{MERGE}(s_i, t_i)$ 
5: end for
6: for  $i = n + 1$  to  $n + m$  do
7:   if  $\text{FIND}(s_i) = \text{FIND}(t_i)$  then
8:     return UNSATISFIABLE
9:   end if
10: end for
11: return SATISFIABLE

```

Steps:

1. **Construct the DAG:** Build the DAG representation of all terms in F
2. **Process equalities:** For each equality $s_i = t_i$ in F^+ , call $\text{MERGE}(s_i, t_i)$ to merge the equivalence classes and propagate congruence
3. **Check disequalities:** For each disequality $s_i \neq t_i$ in F^- , check if s_i and t_i are in the same equivalence class (i.e., $\text{FIND}(s_i) = \text{FIND}(t_i)$)
 - If yes, return UNSATISFIABLE
4. **Return result:** If no conflicts found, return SATISFIABLE

17.4.2 Complexity

Complexity Analysis

Let n be the number of nodes in the DAG. The complexity of the congruence closure algorithm is $O(n^2)$ in the worst case, where the quadratic factor comes from the potential number of congruence propagations that may occur during the MERGE operations.

18 Worked Examples of Congruence Closure

In this section, we present detailed worked examples demonstrating the execution of the congruence closure algorithm on concrete formulas. These examples illustrate the DAG construction, the initialization of node fields, and the step-by-step execution of the **MERGE** operations.

18.1 Example 1: Simple Commutativity

Example 18.1 (Congruence Closure with Commutativity). Consider the formula without quantifiers:

$$F = \{f(x, y) = f(y, x)\} \cup \{f(a, y) = f(y, a)\}$$

where a is a constant symbol.

Step 1: Identify the set of subterms

The set of all subterms appearing in F is:

$$S_F = \{x, y, a, f(x, y), f(y, x), f(a, y), f(y, a)\}$$

Step 2: Construct the DAG

We construct a DAG where each node represents a unique subterm. Each node has the following fields:

- **id**: A unique integer identifier
- **fn**: The function symbol (or variable/constant name) as a string
- **args**: List of identifiers of argument nodes (empty for variables/constants)
- **find**: Initially set to the node's own id
- **ccpar**: Set of parent node identifiers (initially contains all nodes that have this node as an argument)

The DAG structure can be visualized as:

$$\begin{array}{cccc} f_3 & f_4 & f_5 & f_6 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ (x_0, y_1) & (y_1, x_0) & (a_2, y_1) & (y_1, a_2) \end{array}$$

Step 3: Initialize node fields

Initial DAG Node Configuration

```

Node f_3:  id = 3, fn = "f", args = [0, 1], find = 3, ccpar = {}
Node x_0:  id = 0, fn = "x", args = [], find = 0, ccpar = {3, 4}
Node f_4:  id = 4, fn = "f", args = [1, 0], find = 4, ccpar = {}
Node y_1:  id = 1, fn = "y", args = [], find = 1, ccpar = {3, 4, 5, 6}
Node f_5:  id = 5, fn = "f", args = [2, 1], find = 5, ccpar = {}
Node a_2:  id = 2, fn = "a", args = [], find = 2, ccpar = {5, 6}
Node f_6:  id = 6, fn = "f", args = [1, 2], find = 6, ccpar = {}

```

Key observations about initialization:

- Each node's `find` field initially points to itself
- The `ccpar` field of each leaf node (variables/constants) contains the ids of all function nodes that have it as an argument
- Function nodes initially have empty `ccpar` sets

Step 4: Process the equality $f(x, y) = f(y, x)$

We execute `MERGE(3, 4)` corresponding to the equality $f(x, y) = f(y, x)$:

1. Check if `FIND(3) ≠ FIND(4)`: YES (they are 3 and 4 respectively)
2. Save parent sets: $P_1 = \text{CCPAR}(3) = \emptyset$, $P_2 = \text{CCPAR}(4) = \emptyset$
3. Execute `UNION(3, 4)`:
 - Set node 4's `find` field to 3
 - Merge `ccpar` sets (both empty in this case)
4. Check Cartesian product $P_1 \times P_2 = \emptyset \times \emptyset = \emptyset$
5. No further merges needed

After this operation, nodes f_3 and f_4 are in the same equivalence class.

Step 5: Check the disequality (if any)

In this example, we only have equalities. The formula $f(a, y) = f(y, a)$ represents another equality that could be processed similarly. Since $f(a, y)$ and $f(y, a)$ (nodes 5 and 6) remain in distinct equivalence classes after processing the first equality, the formula is **SATISFIABLE**—there is no contradiction.

18.2 Example 2: Cyclic Function Application with Disequality

Example 18.2 (Detecting Unsatisfiability through Congruence Closure). Consider the formula:

$$F = \{f^3(a) = f^5(a)\} \cup \{f(a) \neq a\}$$

where we use the notation $f^n(a)$ to denote n applications of function f to constant a . Explicitly:

- $f^3(a) = f(f(f(a)))$
- $f^5(a) = f(f(f(f(f(a)))))$

Step 1: Identify the set of subterms

The set of all subterms is:

$$S_F = \{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}$$

Step 2: Construct the DAG

We build a chain-like DAG where each node represents successive applications of f :

$$f^5(a) \rightarrow f^4(a) \rightarrow f^3(a) \rightarrow f^2(a) \rightarrow f(a) \rightarrow a$$

Step 3: Initialize node fields

Initial DAG Configuration

```
Node 5: id = 5, fn = "f", args = [4], find = 5, ccpar = {}
Node 4: id = 4, fn = "f", args = [3], find = 4, ccpar = {5}
Node 3: id = 3, fn = "f", args = [2], find = 3, ccpar = {4}
Node 2: id = 2, fn = "f", args = [1], find = 2, ccpar = {3}
Node 1: id = 1, fn = "f", args = [0], find = 1, ccpar = {2}
Node 0: id = 0, fn = "a", args = [], find = 0, ccpar = {1}
```

Step 4: Process the equality $f^3(a) = f^5(a)$

The equality $f^3(a) = f^5(a)$ implies that after three applications of f , we get the same result as after five applications. This creates a cycle in the equivalence structure.

We execute `MERGE(3, 5)`. However, the congruence closure algorithm will propagate this equality through the structure, ultimately establishing relationships between multiple nodes.

Logical implications of the equality:

From $f^3(a) = f^5(a)$, by applying the congruence rule repeatedly, we can derive:

$$\begin{aligned} f^3(a) &= f^5(a) \\ f^4(a) &= f^6(a) = f(f^5(a)) = f(f^3(a)) \\ f^2(a) &= f^4(a) \quad (\text{by applying } f^{-1} \text{ conceptually}) \\ f(a) &= f^3(a) \\ a &= f^2(a) \end{aligned}$$

More precisely, the algorithm establishes these equivalences through congruence propagation.

Step 5: Detailed execution trace

We now trace the execution of `MERGE(3, 5)` and observe how congruence propagates:

Execution Trace**Call: MERGE(3, 5)**

Since $f^3(a) = f^5(a)$, we need to merge nodes 3 and 5.

1. $\text{FIND}(3) = 3, \text{FIND}(5) = 5$ (different, so proceed)
2. $P_1 = \text{CCPAR}(3) = \{4\}, P_2 = \text{CCPAR}(5) = \emptyset$
3. $\text{UNION}(3, 5)$: merge the equivalence classes
4. Check $P_1 \times P_2 = \{4\} \times \emptyset = \emptyset$ (no propagation yet)

However, the equality $f^3(a) = f^5(a)$ implies $f^2(a) = f^4(a)$ by the congruence property. This triggers further merges.

Call: MERGE(3, 0) (establishing $f^3(a) = a$)

1. $P_1 = \text{CCPAR}(3) = \{4\}, P_2 = \text{CCPAR}(0) = \{1\}$
2. $\text{UNION}(3, 0)$
3. Check $P_1 \times P_2 = \{4\} \times \{1\} = \{(4, 1)\}$
4. $\text{CONGRUENT}(4, 1)$?
 - Same symbol: YES ($f = f$)
 - Same arity: YES (both unary)
 - Arguments equivalent: $\text{FIND}(3) = \text{FIND}(0)$ after union: YES
5. Result: TRUE $\Rightarrow$ call $\text{MERGE}(4, 1)$

Call: MERGE(4, 1) (establishing $f^4(a) = f(a)$)

1. $P_1 = \text{CCPAR}(4) = \{5\}, P_2 = \text{CCPAR}(1) = \{2\}$
2. $\text{UNION}(4, 1)$
3. Check $P_1 \times P_2 = \{5\} \times \{2\} = \{(5, 2)\}$
4. $\text{CONGRUENT}(5, 2)$?
 - Same symbol: YES
 - Same arity: YES
 - Arguments equivalent: $\text{FIND}(4) = \text{FIND}(1)$: YES
5. Result: TRUE $\Rightarrow$ call $\text{MERGE}(5, 2)$

Call: MERGE(5, 2) (establishing $f^5(a) = f^2(a)$)

1. $P_1 = \text{CCPAR}(5) = \emptyset, P_2 = \text{CCPAR}(2) = \{3\}$
2. $\text{UNION}(5, 2)$
3. Check $P_1 \times P_2 = \emptyset \times \{3\} = \emptyset$
4. No further propagation

The congruence closure process continues, eventually establishing that all nodes $\{0, 1, 2, 3, 4, 5\}$ are in the same equivalence class, meaning:

$$a = f(a) = f^2(a) = f^3(a) = f^4(a) = f^5(a)$$

Step 6: Check the disequality $f(a) \neq a$

After processing all equalities through congruence closure, we check the disequality constraint:

- We have $f(a) \neq a$ in F^-
- Check: $\text{FIND}(1) = \text{FIND}(0)$?
- After congruence closure: YES (they are in the same equivalence class)
- **Conclusion:** The disequality is violated!

Result: The formula is **UNSATISFIABLE**. The equality $f^3(a) = f^5(a)$ forces a cycle that ultimately implies $f(a) = a$, which contradicts the disequality constraint $f(a) \neq a$.

19 Complexity Improvements and Optimizations

The basic congruence closure algorithm presented earlier has complexity $O(n^2)$ for n nodes in the DAG. However, several optimizations have been developed to improve the practical and theoretical performance of the algorithm.

19.1 Historical Development

19.1.1 Nelson-Oppen Algorithm (1980)

The original congruence closure algorithm by Nelson and Oppen (1980) operates on a graph $G = (V, E)$ where:

- $|V| = n$ (number of nodes/vertices)
- $|E| = e$ (number of edges)

Nelson-Oppen Complexity

The Nelson-Oppen algorithm has complexity $O(e^2)$ for $O(n)$ calls to MERGE.

This quadratic complexity in the number of edges can be prohibitive for large formulas with many equalities.

19.1.2 Downey-Sethi-Tarjan Improvements (1980-1982)

Downey, Sethi, and Tarjan developed several optimizations that significantly improved the complexity of congruence closure algorithms.

Improved Complexity

The Downey-Sethi-Tarjan optimizations reduce the complexity to $O(e \log e)$ for $O(n)$ calls to MERGE.

This represents a substantial improvement, especially for sparse graphs where e is much smaller than n^2 .

19.2 Key Optimizations

The improvements introduced by Downey, Sethi, and Tarjan focus on two main aspects of the algorithm:

19.2.1 Optimization 1: Avoiding Recursion in FIND

Original recursive implementation:

```
let rec FIND i =
  let n = NODE i in
  if n.find = i then i
  else FIND n.find
```

Optimized non-recursive implementation:

```
let FIND i = (NODE i).find
```

Note. This optimization is correct only if the UNION operation is modified appropriately. When node i_1 becomes the representative of the merged equivalence class, the **find** field of **all nodes** in i_2 's class must be updated to point directly to i_1 .

This eliminates the need for path traversal during **FIND** operations, reducing the cost from $O(h)$ (where h is the height of the union-find tree) to $O(1)$.

19.2.2 Optimization 2: Union by Size Heuristics

When merging two equivalence classes via **UNION**(i_1, i_2), we must choose which representative becomes the representative of the merged class. Two effective heuristics are:

Union Heuristics

Heuristic 1: Union by Equivalence Class Size

Choose as the representative of the new class the representative of the **larger equivalence class**:

- Compare $||i_1||$ and $||i_2||$ (sizes of the equivalence classes)
- Make the representative of the larger class the new representative

Heuristic 2: Union by CCPAR Size

Choose as the representative the node whose **ccpar** field has **more members**:

- Compare $|i_1.\text{ccpar}|$ and $|i_2.\text{ccpar}|$
- Make the representative with the larger **ccpar** set the new representative

Rationale: These heuristics ensure that the union-find tree remains balanced, preventing degenerate cases where the tree becomes a long chain. By always attaching the smaller structure to the larger one, we maintain logarithmic depth in the union-find structure.

19.3 Implementation Note

The **CCPAR** function must retrieve the **ccpar** set of the representative node:

```
let CCPAR i = (NODE (FIND i)).ccpar
```

This ensures that we always access the **ccpar** information from the current representative of the equivalence class, which is essential for correctness after multiple **UNION** operations.

19.4 Practical Impact

These optimizations have significant practical impact:

- **Reduced time complexity:** From $O(e^2)$ to $O(e \log e)$
- **Better cache locality:** Non-recursive **FIND** reduces function call overhead
- **Balanced structures:** Union heuristics prevent worst-case tree depths
- **Scalability:** Enables congruence closure on much larger formulas

These improvements make congruence closure practical for real-world applications in automated theorem proving, program verification, and satisfiability modulo theories (SMT) solvers.

Part V

Advanced Topics in Congruence Closure

20 Congruence Closure Implementation

20.1 DAG-Based Implementation

The congruence closure algorithm is implemented on a Directed Acyclic Graph (DAG) representation of the terms. We previously presented the pseudocode for this implementation.

20.2 Bradley-Miller Algorithm (2007)

We also studied the Bradley-Miller algorithm from 2007, which appears in a paper by Nelson and Oppen. This algorithm has complexity $O(e^2)$ assuming $O(n)$ calls to **MERGE**, where:

- n is the number of vertices (nodes) in the DAG
- e is the number of edges in the DAG

20.3 Downey-Sethi-Tarjan Improvements (1980)

We also discussed the Downey-Sethi-Tarjan paper from 1980, which achieves improved complexity of $O(e \log e)$ under the same assumptions.

The two key ingredients for achieving this improved complexity are:

- A non-recursive version of the **FIND** operation
- Selection of the representative of the new class in **UNION** based on the size of `n.ccpair`

20.4 Forbidden Set Optimization

The forbidden set feature was implemented in 2005 by Oppen, Detlefs, and Saxe in the context of the “Simplify” theorem prover.

20.4.1 Problem Structure

During congruence closure, we solve problems of the form:

$$F = \underbrace{\{s_i = t_i\}}_{F^+} \cup \underbrace{\{s_i \neq t_i\}}_{F^-}$$

where i ranges from 1 to n for positive equalities and from $n + 1$ to $n + m$ for negative equalities.

The process involves:

1. Constructing the set of terms S_F
2. Building the DAG representation
3. Computing the congruence closure of F^+
4. For each disequality $s_i \neq t_i$ (where $n + 1 \leq i \leq n + m$), checking if the nodes representing s_i and t_i are in the same equivalence class

Let u_i be the node representing s_i and v_i be the node representing t_i . If $\text{FIND}(u_i) = \text{FIND}(v_i)$, then return **UNSATISFIABLE**.

20.4.2 Forbidden Set Mechanism

The forbidden set mechanism enhances the basic algorithm by adding early detection of unsatisfiability:

Forbidden Set Implementation

Algorithm enhancement:

1. Extend the node data structure with an additional field:
`forbidden: Id set`
2. For each disequality $s_i \neq t_i$ (where $n + 1 \leq i \leq n + m$):
 - Add the node representing s_i to the forbidden set of the node representing t_i
 - Add the node representing t_i to the forbidden set of the node representing s_i
3. Modify the MERGE function:


```

MERGE i_1 i_2:
  if FIND i_1 != FIND i_2 then
    ... (standard merge operations)
    if i_1 in n_1.forbidden or i_2 in n_2.forbidden then
      return UNSATISFIABLE
      
```

where n_1 is the node for i_1 and n_2 is the node for i_2 .

This optimization allows for early termination when a merge operation would violate a disequality constraint, potentially avoiding unnecessary computation.

20.5 Summary of Congruence Closure

The congruence closure algorithm provides a complete and sound decision procedure for the quantifier-free fragment of the theory of equality. The algorithm:

- Constructs the smallest congruence relation containing all positive equalities
- Checks whether any negative equality is violated by this congruence
- Returns **SATISFIABLE** if no violations are found, **UNSATISFIABLE** otherwise

The various optimizations (Bradley-Miller, Downey-Sethi-Tarjan, forbidden sets) improve the practical efficiency while maintaining correctness.

21 The Theory of Lists

The theory of lists represents a special case of parametric theories of recursive data structures, providing a formal framework for reasoning about list structures and operations.

21.1 Signatures for List Theories

List Theory Signatures

The theory of lists can be formalized using two different signatures:

- **Signature** Σ_{CONS} (non-empty lists):

$$\Sigma_{CONS} = \{\text{cons}, \text{car}, \text{cdr}, \simeq, \text{atom}\}$$

- **Signature** Σ_{NIL} (possibly empty lists):

$$\Sigma_{NIL} = \{\text{cons}, \text{car}, \text{cdr}, \simeq, \text{nil}\}$$

Note: We will return to the Σ_{NIL} signature later in our discussion.

21.2 Axioms of the List Theory

The axioms for the Σ_{CONS} theory are denoted A_{CONS} and consist of congruence axioms together with selector-over-constructor axioms.

21.2.1 Congruence Axioms

The congruence axioms establish that equality is preserved under list operations:

Equality Axioms

$$\begin{aligned} \forall x (x \simeq x) & \quad (\text{Reflexivity}) \\ \forall x \forall y (x \simeq y \rightarrow y \simeq x) & \quad (\text{Symmetry}) \\ \forall x \forall y \forall z ((x \simeq y \wedge y \simeq z) \rightarrow x \simeq z) & \quad (\text{Transitivity}) \end{aligned}$$

Function Congruence Axioms

$$\begin{aligned} \forall x \forall y (x \simeq y \rightarrow \text{car}(x) \simeq \text{car}(y)) \\ \forall x \forall y (x \simeq y \rightarrow \text{cdr}(x) \simeq \text{cdr}(y)) \\ \forall x_1 \forall x_2 \forall y_1 \forall y_2 ((x_1 \simeq x_2 \wedge y_1 \simeq y_2) \rightarrow \text{cons}(x_1, y_1) \simeq \text{cons}(x_2, y_2)) \end{aligned}$$

Predicate Congruence Axiom

$$\forall x \forall y (x \simeq y \rightarrow (\text{atom}(x) \leftrightarrow \text{atom}(y)))$$

21.2.2 Selector-over-Constructor Axioms

These axioms define the fundamental properties of the list constructors and selectors:

Constructor-Selector Axioms

$$\begin{aligned} \forall x \forall y (\text{car}(\text{cons}(x, y)) &= x) \\ \forall x \forall y (\text{cdr}(\text{cons}(x, y)) &= y) \\ \forall x \forall y (\neg \text{atom}(\text{cons}(x, y))) & \\ \forall x (\text{cons}(\text{car}(x), \text{cdr}(x)) &= x) \quad (\text{provided } \neg \text{atom}(x)) \end{aligned}$$

These axioms formalize the basic operations on lists:

- $\text{cons}(x, y)$ constructs a new list with head x and tail y
- $\text{car}(x)$ returns the head (first element) of list x
- $\text{cdr}(x)$ returns the tail (remaining elements) of list x
- $\text{atom}(x)$ tests whether x is an atomic element (not a list)

The final axiom states that applying car and cdr to a non-atomic list and then reconstructing with cons recovers the original list, provided the list is not atomic.

21.3 Validity Examples in List Theory

Consider the following formula in the theory $\mathcal{T}_{CONS} = \langle \Sigma_{CONS}, A_{CONS} \rangle$:

Example 21.1 (Validity in List Theory). **Input formula:**

$$F : \quad \text{car}(x) = y \wedge \text{cdr}(x) = z \rightarrow x = \text{cons}(y, z)$$

Question: Is F $\mathcal{T}_{CONS}$ -valid?

Answer: No, because x could be interpreted as an atom.

However, we can modify the formula to ensure validity:

$$F' : \quad \neg \text{atom}(x) \wedge \text{car}(x) = y \wedge \text{cdr}(x) = z \rightarrow x = \text{cons}(y, z)$$

By the selector-over-constructor axioms, we have $\text{cons}(\text{car}(x), \text{cdr}(x)) = x$, which implies $\text{cons}(y, z) = x$. Therefore, F' is $\mathcal{T}_{CONS}$ -valid.

21.4 The Theory of Possibly Empty Lists

We now consider the theory of possibly empty lists, which extends the previous theory to handle the empty list case.

Definition 21.1 (Theory of Possibly Empty Lists). **Signature:**

$$\Sigma_{NIL} = \{\text{cons}, \text{car}, \text{cdr}, \simeq, \text{nil}\}$$

Axioms A_{NIL} :

The axiom set includes the congruence axioms from A_{CONS} plus additional axioms for the empty list:

$$\begin{aligned} n_1 : \quad & \forall x \forall y (\text{cons}(x, y) \neq \text{nil}) \\ n_2 : \quad & \forall x (x \neq \text{nil} \rightarrow \text{cons}(\text{car}(x), \text{cdr}(x)) = x) \end{aligned}$$

Axiom n_1 ensures that constructed lists are never empty, while axiom n_2 states that for non-empty lists, the reconstruction property holds.

21.5 Generalization to Recursive Data Structures

The theory of lists can be generalized to arbitrary recursive data structures.

Definition 21.2 (Recursive Data Structures (RDS)). Consider a signature $\Sigma_{CONS} = \text{RDS}_k$ for a recursive data structure with:

- One **constructor** C of arity k
- k **selectors** $s_1, \dots, s_k$

The corresponding axioms would be:

$$\begin{aligned} & \forall x_1 \dots \forall x_k (s_i(C(x_1, \dots, x_k)) = x_i) && \text{for each } i = 1, \dots, k \\ & \forall x_1 \dots \forall x_k (\neg \text{atom}(C(x_1, \dots, x_k))) \\ & \forall x (\neg \text{atom}(x) \rightarrow x = C(s_1(x), \dots, s_k(x))) \end{aligned}$$

These axioms generalize the list theory to arbitrary tree-like recursive data structures where:

- The constructor creates structured data from components
- The selectors extract the components from structured data
- The atom predicate distinguishes between atomic and structured data

This framework provides a foundation for reasoning about complex recursive data types in automated reasoning systems.

Part VI

Theory of Lists and Recursive Data Structures

22 Decision Procedure for Equality and Lists

We analyse the decision procedure for the union of the quantifier-free fragments of the theories of equality and lists. The extensionality axiom states that two constructed lists are equal whenever their components coincide:

$$\forall x_1, x_2, y_1, y_2 \left(x_1 \simeq x_2 \wedge y_1 \simeq y_2 \rightarrow \text{cons}(x_1, y_1) \simeq \text{cons}(x_2, y_2) \right).$$

22.1 Problem Setting

Let F be a quantifier-free formula over the signature Σ_{CONS} . We first convert F into disjunctive normal form,

$$F \equiv C_1 \vee C_2 \vee \dots \vee C_k,$$

where every C_i is a conjunction of literals. The satisfiability question is reduced to each conjunction separately: for a fixed C_i we write S for the associated set of literals over Σ_{CONS} .

Note. Reminder: $P(\bar{t})f_1(\bar{t}) = [\text{dot}]$ while $P(\bar{t})f_P(\bar{t}) \neq [\text{dot}]$.

Our strategy enriches the standard Congruence Closure (CC) algorithm with the axioms specific to lists, yielding a decision procedure for $\mathcal{T}_{\text{CONS}} \cup \mathcal{T}_=$.

22.2 Structure of the Literal Set

Each conjunction S can be written as

$$S = \{s_i = t_i\}_{i=1}^n \cup \{s_i \neq t_i\}_{i=n+1}^{n+m} \cup \{\text{atom}(u_i)\}_{i=n+m+1}^{n+m+k} \cup \{\neg \text{atom}(u_i)\}_{i=n+m+1}^{n+m+k}.$$

Congruence axioms are already handled by CC. The remaining axioms of the list theory are:

$$\begin{aligned} \forall x \forall y \text{ car}(\text{cons}(x, y)) &= x, \\ \forall x \forall y \text{ cdr}(\text{cons}(x, y)) &= y, \\ \forall x \forall y \neg \text{atom}(\text{cons}(x, y)), \\ \forall x \neg \text{atom}(x) \rightarrow x &= \text{cons}(\text{car}(x), \text{cdr}(x)). \end{aligned}$$

22.3 Eliminating Negative Atom Literals

We preprocess S by replacing every $\neg \text{atom}(u_i)$ with the equality

$$u_i = \text{cons}(u_i^{(1)}, u_i^{(2)}),$$

where $u_i^{(1)}$ and $u_i^{(2)}$ are fresh free variables. This accounts for the reconstruction axiom directly: $\text{car}(u_i) = u_i^{(1)}$ and $\text{cdr}(u_i) = u_i^{(2)}$. After preprocessing we obtain

$$F = \{s_i = t_i\}_{i=1}^n \cup \{s_i \neq t_i\}_{i=n+1}^{n+m} \cup \{\text{atom}(u_i)\}_{i=n+m+1}^{n+m+k}$$

and denote by S_F the set of all terms occurring in F .

22.4 Extending the Congruence Closure Graph

We build the directed acyclic graph (DAG) for S_F as in the pure equality case and then add the list axioms structurally. For every node whose function symbol is **cons** we introduce two new nodes, representing $\text{car}(t)$ and $\text{cdr}(t)$, where t is the term stored in the original node. We enforce the selector axioms by executing

$$\text{MERGE}(n_1, n.\text{args}[0]) \quad \text{and} \quad \text{MERGE}(n_2, n.\text{args}[1]),$$

which merges the newly added nodes with the appropriate arguments. No recursion occurs because the nodes are fresh. At this point we run the CC algorithm exactly as we would for the theory of equality.

22.5 Post-processing for Atom Constraints

The axiom $\forall x \forall y \neg \text{atom}(\text{cons}(x, y))$ is handled after CC terminates. For each i with $n + m + 1 \leq i \leq n + m + k$, if

$$\text{FIND}(u_i) = \text{FIND}(v)$$

for some node v representing a **cons** term, we conclude that S is unsatisfiable: u_i would be equal to a constructed list, contradicting the atom predicate.

22.6 Example

Example 22.1. Consider

$$F = \{\text{car}(x) = \text{car}(y), \text{cdr}(x) = \text{cdr}(y), f(x) \neq f(y), \neg \text{atom}(x), \neg \text{atom}(y)\}.$$

From the negative atom literals we derive

$$x = \text{cons}(\text{car}(x), \text{cdr}(x)) \quad \text{and} \quad y = \text{cons}(\text{car}(y), \text{cdr}(y)).$$

Congruence propagates the selector equalities, yielding $x = y$. This contradicts $f(x) \neq f(y)$, so F is unsatisfiable in $\mathcal{T}_{\text{CONS}} \cup \mathcal{T}_{=}$.

Part VII

Satisfiability Modulo Theories and Theorem Proving

23 Overview of SMT and ATP

23.1 Satisfiability Modulo Theories

The topic of Satisfiability Modulo Theories (SMT) focuses on procedures to determine satisfiability in specific first-order logic theories. The course covers the quantifier-free fragments (QFF) of several important theories:

- Theory of Equality
- Theory of CONS (Lists)
- Theory of Arrays
- Combination of Theories
- Integration with SAT solvers

23.2 Automated Theorem Proving

In contrast to SMT, Automated Theorem Proving (ATP) deals with full first-order logic:

- Admits quantifiers
- Equality as the only defined symbol
- Uses semidecision procedures

24 Reduction to Clausal Form

The reduction of first-order formulas to clausal form is a fundamental transformation necessary for both SMT and ATP. This process converts arbitrary first-order formulas into a standardized form suitable for automated reasoning procedures.

24.1 Basic Logical Equivalences

The first step involves eliminating certain connectives using standard logical equivalences:

$$\begin{aligned}
 F \leftrightarrow G &\equiv (F \rightarrow G) \wedge (G \rightarrow F) \\
 F \rightarrow G &\equiv \neg F \vee G \\
 \neg(F \wedge G) &\equiv \neg F \vee \neg G \\
 \neg(F \vee G) &\equiv \neg F \wedge \neg G
 \end{aligned}$$

24.2 Negation Normal Form

Definition 24.1 (Negation Normal Form). A formula is in **Negation Normal Form (NNF)** if the only connectives are $\vee$, $\wedge$, and $\neg$, and all occurrences of negation apply directly to atoms.

To achieve NNF when quantifiers are present, we push negations through quantifiers using the following equivalences:

$$\neg \forall x F \equiv \exists x \neg F$$

$$\neg \exists x F \equiv \forall x \neg F$$

These transformations are analogous to De Morgan's laws. The universal quantifier can be viewed as a potentially infinite conjunction, while the existential quantifier corresponds to a potentially infinite disjunction.

24.3 Clausal Form and CNF

Clausal Form and CNF

Definition 24.2 (Clause). A **clause** is a disjunction of literals.

Definition 24.3 (Conjunctive Normal Form). A formula is in **Conjunctive Normal Form (CNF)** if it is a conjunction of clauses (i.e., a conjunction of disjunctions of literals).

Definition 24.4 (Clausal Form). In first-order logic, **clausal form** is a set of clauses where all variables are implicitly universally quantified. There are no free variables and no existentially quantified variables.

24.4 Skolemization

Skolemization is the process of eliminating existential quantifiers by introducing new symbols.

24.4.1 Outermost Existential Quantifiers

For a formula $\exists x F$ where the existential quantifier is outermost, we replace all occurrences of x with a fresh constant symbol a :

$$\exists x F \rightsquigarrow F[x \leftarrow a]$$

where a is a new **Skolem constant**.

Example 24.1.

$$\exists x [(P(x) \wedge P(a)) \vee (R(x) \rightarrow Q(x))]$$

Introducing the fresh constant b for x , we obtain:

$$(P(b) \wedge P(a)) \vee (R(b) \rightarrow Q(b))$$

For multiple consecutive existential quantifiers:

$$\begin{aligned} \exists x \exists y R(x, y) &\rightsquigarrow \exists y R(a, y) \\ &\rightsquigarrow R(a, b) \end{aligned}$$

where a and b are new constant symbols.

24.4.2 Nested Existential Quantifiers

For a formula where an existential quantifier appears within the scope of universal quantifiers:

$$\forall y_1 \forall y_2 \cdots \forall y_n \exists x F \quad \text{for } n \geq 1$$

we replace x with a fresh function applied to all universally quantified variables in whose scope it appears:

$$\forall \bar{y} \exists x F \rightsquigarrow \forall \bar{y} F[x \leftarrow g(\bar{y})]$$

where g is a new **Skolem function symbol**.

Definition 24.5 (Skolem Symbols). • A **Skolem constant** is a fresh constant symbol introduced to replace an outermost existential quantifier

- A **Skolem function symbol** is a fresh function symbol introduced to replace a nested existential quantifier
- **Skolem terms** are the terms constructed using these symbols (e.g., a , b , $g(\bar{y})$)

24.5 Dropping Universal Quantifiers

After Skolemization, all remaining quantifiers are universal. These can be dropped, with the understanding that all variables are implicitly universally quantified, which is exactly the requirement for clausal form.

24.6 Distribution to CNF

The final step applies distributivity to transform the formula into a conjunction of disjunctions:

$$F \vee (G \wedge H) \equiv (F \vee G) \wedge (F \vee H)$$

The result is a set of clauses in clausal form.

24.7 Complete Example

Example 24.2. Consider the formula:

$$\forall x (P(x) \rightarrow [\forall y (P(y) \rightarrow P(f(x, y))) \wedge \neg \forall y (Q(x, y) \rightarrow P(y))])$$

Step 1: Eliminate implications

$$\forall x (\neg P(x) \vee [\forall y (\neg P(y) \vee P(f(x, y))) \wedge \exists y \neg (\neg Q(x, y) \vee P(y))])$$

Step 2: Push negations inward (NNF)

$$\forall x (\neg P(x) \vee [\forall y (\neg P(y) \vee P(f(x, y))) \wedge \exists y (Q(x, y) \wedge \neg P(y))])$$

Step 3: Standardize variables apart

$$\forall x (\neg P(x) \vee [\forall y_1 (\neg P(y_1) \vee P(f(x, y_1))) \wedge \exists y_2 (Q(x, y_2) \wedge \neg P(y_2))])$$

Step 4: Skolemization

The existential quantifier $\exists y_2$ appears in the scope of $\forall x$ and $\forall y_1$. We replace y_2 with $g(x, y_1)$ where g is a fresh function symbol:

$$\forall x \forall y_1 (\neg P(x) \vee [(\neg P(y_1) \vee P(f(x, y_1))) \wedge Q(x, g(x, y_1)) \wedge \neg P(g(x, y_1))])$$

Alternative Skolemization: If we use $y_2 \leftarrow g(x)$ (depending only on x), we obtain a simpler form.

Step 5: Drop universal quantifiers and distribute

The formula is already in a form that can be converted to clauses by distribution.

Part VIII

Theory of Arrays

25 The Theory of Arrays

We now investigate the theory of arrays, following our study of the theory of equality and the theory of lists.

Theory of Arrays

Definition 25.1 (Theory of Arrays). The theory of arrays is defined as:

$$\mathcal{T}_A = \langle \Sigma_A, A_A \rangle$$

where Σ_A is the signature and A_A is the axiomatization.

An **array** is a data structure in which we can reach directly every value contained in the array through indexing. Arrays support two fundamental operations:

- **Read:** $a[i]$ (also called **select**)
- **Write:** $a[i] := w$ (also called **store**)

25.0.1 Signature of the Theory of Arrays

Signature of the Theory of Arrays

The signature of the theory of arrays is:

$$\Sigma_A = \{\text{select}, \text{store}, \simeq\}$$

In general, a signature of a theory $\Sigma_A = \langle \mathcal{S}_A, \mathcal{F}_A \rangle$ consists of:

- $\mathcal{S}_A$: the set of sorts
- $\mathcal{F}_A$: the set of function symbols

For the theory of arrays, we have:

$$\mathcal{S}_A = \{\text{ARRAY}, \text{INDEX}, \text{VALUE}\}$$

These are all distinct sorts. This means that in reality we have three equality relations:

$$\simeq_{\text{ARRAY}}, \simeq_{\text{INDEX}}, \simeq_{\text{VALUE}}$$

25.0.2 Function Symbols: Select and Store

For function symbols in general, we need to know not only the arity, but also the sorts involved. A function f has type:

$$f : s_1 \times s_2 \times \cdots \times s_k \rightarrow s$$

For example, addition has type $+: \text{int} \times \text{int} \rightarrow \text{int}$.

The Select Function

$$\text{select} : \text{ARRAY} \times \text{INDEX} \rightarrow \text{VALUE}$$

We write $a[i]$ as $\text{select}(a, i)$.

The Store Function

$$\text{store} : \text{ARRAY} \times \text{INDEX} \times \text{VALUE} \rightarrow \text{ARRAY}$$

The store function takes an array and modifies it by storing a value w at index i , returning a new array.

We write $a[i] := w$ as $\text{store}(a, i, w)$.

Note. In general, it may be true that $a \preceq \text{store}(a, i, w)$, but it could also be that $a \not\preceq \text{store}(a, i, w)$ since we are modifying the array.

We can observe that:

- If $\text{select}(a, i) \neq w$, then $a \not\preceq \text{store}(a, i, w)$
- If $\text{select}(a, i) \preceq w$, then $a \preceq \text{store}(a, i, w)$

25.0.3 Axioms of the Theory of Arrays

Now we consider the axioms A_A of the theory of arrays.

Equality Axioms

The basic equality axioms include:

$$\begin{array}{ll} \forall x (x \preceq x) & \text{(Reflexivity)} \\ \forall x \forall y (x \preceq y \rightarrow y \preceq x) & \text{(Symmetry)} \\ \forall x \forall y \forall z ((x \preceq y \wedge y \preceq z) \rightarrow x \preceq z) & \text{(Transitivity)} \end{array}$$

Congruence Axioms

$$\begin{array}{l} \forall x_1 \forall x_2 \forall y_1 \forall y_2 ((x_1 \preceq x_2 \wedge y_1 \preceq y_2) \rightarrow \text{select}(x_1, y_1) \preceq \text{select}(x_2, y_2)) \\ \forall x_1 \forall x_2 \forall y_1 \forall y_2 \forall z_1 \forall z_2 ((x_1 \preceq x_2 \wedge y_1 \preceq y_2 \wedge z_1 \preceq z_2) \\ \rightarrow \text{store}(x_1, y_1, z_1) \preceq \text{store}(x_2, y_2, z_2)) \end{array}$$

Select-over-Store Axioms

$$\begin{array}{l} \forall x \forall y \forall z (\text{select}(\text{store}(x, y, z), y) \preceq z) \\ \forall x \forall y_1 \forall y_2 \forall z (y_1 \neq y_2 \rightarrow \text{select}(\text{store}(x, y_1, z), y_2) \preceq \text{select}(x, y_2)) \end{array}$$

Note. The select-over-store axioms are analogous to those in the theory of lists.

Extensionality Axiom

$$\forall x_1 \forall x_2 ((\forall y (\text{select}(x_1, y) \simeq \text{select}(x_2, y))) \rightarrow x_1 \simeq x_2)$$

This axiom can be written as a bi-implication:

$$\forall x_1 \forall x_2 (x_1 \simeq x_2 \leftrightarrow (\forall y (\text{select}(x_1, y) \simeq \text{select}(x_2, y))))$$

However, we typically write it as a single implication (from right to left) because the other direction (from left to right) follows from the congruence axioms.

25.1 Validity Example in the Theory of Arrays

Example 25.1 (Validity in Array Theory). Consider the following formula:

$$\phi : \quad \text{select}(a, i) \simeq e \rightarrow \text{store}(a, i, e) \simeq a$$

Question: Is ϕ valid in $\mathcal{T}_A^\infty$ (the theory of arrays with equality)?

Alternative formulation: We can equivalently express the validity as:

$$\text{select}(a, i) \simeq e \rightarrow \forall y (\text{select}(\text{store}(a, i, e), y) \simeq \text{select}(a, y))$$

Proof by contradiction:

Assume the negation of the formula:

I) Suppose:

$$\begin{aligned} &\text{select}(a, i) \simeq e \\ &\neg(\forall y (\text{select}(\text{store}(a, i, e), y) \simeq \text{select}(a, y))) \end{aligned}$$

This is equivalent to:

$$\exists y (\text{select}(\text{store}(a, i, e), y) \not\simeq \text{select}(a, y))$$

Applying Skolemization, we introduce a new constant j :

II)

$$\text{select}(\text{store}(a, i, e), j) \not\simeq \text{select}(a, j)$$

Now we apply the select-over-store axioms to find a contradiction:

- $\text{select}(\text{store}(x, y, z), y) \simeq z$
- $y_1 \not\simeq y_2 \rightarrow \text{select}(\text{store}(x, y_1, z), y_2) \simeq \text{select}(x, y_2)$

We consider two cases:

Case 1: $i \simeq j$

By the first select-over-store axiom:

$$\text{select}(\text{store}(a, i, e), j) \simeq e$$

But from assumption I), we have $\text{select}(a, i) \simeq e$, and since $i \simeq j$, by congruence $\text{select}(a, j) \simeq e$.

Therefore, $\text{select}(\text{store}(a, i, e), j) \simeq \text{select}(a, j)$, which contradicts II).

Case 2: $i \neq j$

By the second select-over-store axiom:

$$\text{select}(\text{store}(a, i, e), j) \simeq \text{select}(a, j)$$

This directly contradicts II).

Since both cases lead to a contradiction, the formula ϕ is valid in $\mathcal{T}_A$.